OpsDeck Documentation

IT Operations & Governance Platform

pixelotes

Copyright © 2024–2026 pixelotes — Elastic License

Table of contents

Copyright © 2024–2026 pixelotes — Elastic License

Copyright © 2024–2026 pixelotes — Elastic License

Copyright © 2024–2026 pixelotes — Elastic License

Copyright © 2024–2026 pixelotes — Elastic License

Copyright © 2024–2026 pixelotes — Elastic License

Copyright © 2024–2026 pixelotes — Elastic License

Copyright © 2024–2026 pixelotes — Elastic License

1. OpsDeck Documentation

OpsDeck is an integrated IT operations and governance platform designed for teams that need operational rigor without enterprise complexity. It serves as the operational system of record for IT departments in regulated environments.

1.1 Sections

	Section	Description
:material-book-open-variant:	**User Guide**	Learn how to use every module — from asset tracking to compliance audits and user access reviews
:material-rocket-launch:	**Deployment**	Get OpsDeck running with Docker, Kubernetes, or a manual install
:material-sitemap:	**Architecture**	System design, data model, dependency choices, and architectural decisions
:material-frequently-asked-questions:	**FAQ**	Common questions about licensing, features, and troubleshooting

1.2 What OpsDeck does

OpsDeck consolidates the essential operational and governance needs of IT teams into a single platform:

Module	What it does
Asset lifecycle	Track hardware from procurement to disposal with maintenance logs and assignment history
Compliance & audits	Continuous monitoring for ISO 27001 / SOC 2 with drift detection and audit snapshots
User access reviews	Automated comparisons between systems to detect orphaned accounts and access mismatches
Service catalog	Map business services to technical dependencies for impact analysis
Risk & security	Risk register, incident management, credentials vault, and audit trails
Vendor management	Supplier tracking, contract lifecycle, subscription renewal forecasting
Policies & training	Policy acknowledgment workflows and training assignment tracking

Built for mid-market IT teams (50–500 employees) in finance, healthcare, SaaS, and other regulated industries.

1.3 License

OpsDeck is distributed under the Elastic License — free to use and modify, with restrictions on offering it as a managed service.

Copyright © 2024–2026 pixelotes — Elastic License

2. User Guide

2.1 User Guide

This section covers how to use OpsDeck day-to-day. Each page documents a functional module with its workflows, screens, and configuration options.

2.1.1 Before you start

If you're new to OpsDeck, begin with Getting Started to understand initial setup, navigation, and core concepts.

2.1.2 Modules at a glance

Area	Modules
IT operations	Assets, peripherals, software, locations, maintenance, disposal
Governance	Frameworks, compliance dashboard, audit snapshots, compliance drift
Access	User access reviews (UAR), users & groups, RBAC
Risk	Risk register, risk assessment, security incidents
Procurement	Suppliers, contracts, subscriptions, purchases, budgets
People	Onboarding/offboarding, hiring pipeline, training, policies
Services	Service catalog, dependency mapping, business impact
Security	Credentials vault, certificates, change management, BCDR
CRM	Contacts, leads, opportunities, campaigns
Platform	Reports, universal search, configuration, data import, REST API

Copyright © 2024–2026 pixelotes — Elastic License

2.2 Getting Started

This guide walks you through your first steps after deploying OpsDeck — from logging in to understanding the core navigation and setting up your organization.

2.2.1 First login

After deployment, open your OpsDeck instance and log in with the admin credentials configured during installation:

- **Default email:** `admin@example.com`
- **Default password:** `admin123`

You'll be prompted to change the default password immediately. If Google OAuth is configured, you can also use "Sign in with Google."

2.2.2 Navigation overview

OpsDeck's interface is organized around a left sidebar with these main sections:

Section	What you'll find
Dashboard	Compliance summary, upcoming renewals, recent activity
Assets	Hardware, peripherals, software, locations, maintenance
Compliance	Frameworks, controls, audits, compliance drift, UAR
Security	Risk register, incidents, security activities, credentials
Vendors	Suppliers, contracts, subscriptions, purchases, budgets
People	Users, groups, onboarding/offboarding, hiring, training
Services	Service catalog, dependency mapping
Reports	Built-in reports, universal search
Administration	Configuration, policies, locations, cost centers

The top bar provides universal search (magnifying glass icon) and quick access to notifications and your user profile.

2.2.3 Core concepts

Before diving in, understand these concepts that appear throughout OpsDeck:

Compliance links. Any entity (asset, policy, service, supplier) can be linked to a framework control as evidence. This is the backbone of compliance management — link evidence continuously, not just during audits.

Tags. Flexible labels that can be attached to most entities for custom categorization and filtering.

Custom properties. Assets, users, and peripherals support custom fields defined at the organization level (Administration → Configuration).

Audit trail. Every creation, update, and deletion is logged with the user, timestamp, and changed fields. This trail is immutable and cannot be deleted from the UI.

2.2.4 Initial configuration checklist

Complete these steps to set up your organization:

Copyright © 2024–2026 pixelotes — Elastic License

1. Organization settings

Navigate to **Administration → Configuration** and set:

- Organization name and logo.
- Default timezone and date format.
- Notification preferences (email, Slack webhook).

2. Locations

Navigate to **Administration → Locations** and create your physical and logical locations:

- Office sites (HQ, branches, data centers).
- Remote locations if tracking remote worker assets.
- Locations support hierarchy (building → floor → room).

3. Cost centers

Navigate to **Administration → Cost Centers** and create departments or budget centers:

- Used for asset assignment, budget allocation, and reporting.
- Map to your organization's financial structure.

4. Users and groups

Navigate to **People → Users** and create your team:

1. Click "Add User" — provide name, email, and role (Admin/Manager/User).
2. Create **Groups** for department-based permission assignment.
3. If using Google OAuth, users can log in immediately with their Google email.

5. Compliance frameworks

Navigate to **Compliance → Frameworks** to set up your compliance requirements:

1. Create or import a framework (ISO 27001, SOC 2, or custom).
2. Review the controls — each one represents a requirement you need evidence for.
3. Start linking existing assets, policies, and services to controls.

6. Suppliers

Navigate to **Vendors → Suppliers** and register your key vendors:

1. Add supplier details, compliance status, and contacts.
2. Link contracts and subscriptions.
3. Set up renewal tracking.

Copyright © 2024–2026 pixelotes — Elastic License

2.2.5 What's next

With the basics configured, explore these workflows:

- **Hardware Assets** — start tracking your asset inventory.
- **Compliance Dashboard** — understand your compliance posture.
- **User Access Reviews** — automate access governance.
- **Data Import (CLI)** — bulk import existing data from CSV.

Copyright © 2024–2026 pixelotes — Elastic License

2.3 Dashboard

The dashboard is your landing page after login, providing an at-a-glance overview of operational and compliance status.

2.3.1 Dashboard widgets

The main dashboard displays:

- **Compliance summary** — per-framework compliance scores with control status breakdown.
- **Upcoming renewals** — subscriptions and contracts approaching their renewal dates.
- **Recent activity** — the latest changes across all modules (asset assignments, incident updates, policy acknowledgments).
- **Open findings** — unresolved UAR findings and risk items requiring attention.
- **Certificate expirations** — certificates approaching expiry.
- **Pending tasks** — onboarding items, training assignments, and policy acknowledgments assigned to you.

2.3.2 Charts

The dashboard uses Chart.js for visual summaries:

- Compliance score trend over time.
- Asset distribution by status and location.
- Risk heat map (impact vs. likelihood).
- Subscription cost breakdown by category.

Copyright © 2024–2026 pixelotes — Elastic License

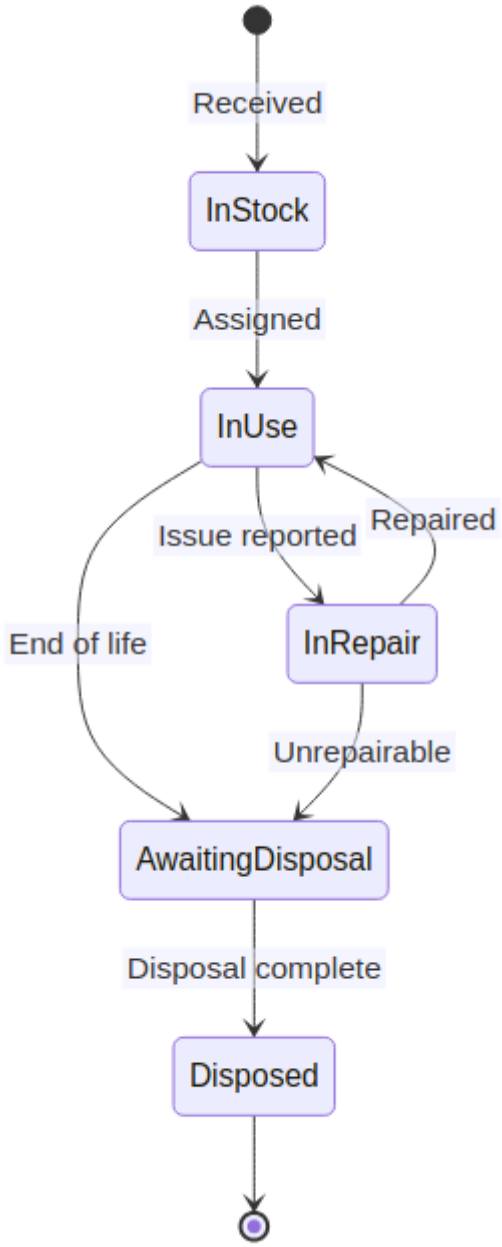
2.4 Assets

2.4.1 Hardware Assets

The asset management module tracks IT hardware from procurement through disposal, with maintenance logging, assignment history, warranty tracking, and compliance linking.

Copyright © 2024–2026 pixelotes — Elastic License

Asset lifecycle states



State	Description
In Stock	Asset received but not yet deployed. Stored at a location.
In Use	Assigned to a user or deployed to a location.
In Repair	Under maintenance. Automatically tracked via maintenance logs.
Awaiting Disposal	Approved for retirement. Pending disposal process.
Disposed	Retired. Record preserved for audit purposes.

Creating an asset

1. Navigate to **Assets** → **Asset List**.
2. Click **Add Asset**.

Copyright © 2024–2026 pixelotes — Elastic License

3. Fill in the required fields: name, asset type, model, serial number.
4. Optional fields: purchase date, cost, warranty expiry, supplier, location.
5. Assign to a user (optional — can be done later).
6. Upload photos or documentation as attachments.
7. Save.

The asset is created in **In Stock** status by default unless assigned to a user.

Assignment and transfers

Assigning an asset to a user:

1. Open the asset detail page.
2. Click **Assign** or edit the assignment field.
3. Select the user. The status changes to **In Use** automatically.
4. An `AssetAssignment` record is created, preserving the assignment history.

Transferring between users:

1. Change the assigned user on the asset detail page.
2. The previous assignment is closed (end date set) and a new one is created.
3. Full assignment history is visible in the asset's timeline.

Warranty tracking

- Set the **Warranty Expiry** date when creating the asset.
- The dashboard and reports show assets with upcoming warranty expirations.
- Notifications can be configured to alert before expiry.

Custom properties

Assets support custom fields defined at the organization level:

1. Navigate to **Administration → Configuration → Custom Fields**.
2. Create a custom field (e.g., "Insurance Policy Number", "BIOS Version").
3. The field appears on all asset forms and detail pages.
4. Custom fields are searchable via universal search.

Bulk operations

From the asset list view:

- **Bulk assign** — assign multiple assets to a user or location.
- **Bulk export** — export filtered asset list as CSV.
- **Bulk import** — use the CLI data import for initial loading.

Copyright © 2024–2026 pixelotes — Elastic License

Compliance linking

Assets are commonly linked to compliance controls as evidence:

1. On the asset detail page, scroll to **Compliance Links**.
2. Click **Link to Control**.
3. Select the framework and control.
4. Add context notes explaining how this asset satisfies the control.

Example: linking a laptop with full-disk encryption to ISO 27001 control A.8.1 (Responsibility for assets).

Copyright © 2024–2026 pixelotes — Elastic License

2.4.2 Peripherals

Track monitors, keyboards, headsets, docking stations, and other peripheral devices with assignment history and custom properties.

Peripheral types

Common peripheral categories: monitors, keyboards, mice, headsets, webcams, docking stations, cables, adapters. Categories are configurable.

Assigning peripherals

1. Navigate to **Assets → Peripherals**.
2. Click **Add Peripheral**.
3. Provide: name, type, serial number, and condition.
4. Assign to a user — creates a `PeripheralAssignment` record tracking who has the peripheral and since when.
5. Transfers between users close the current assignment and create a new one, preserving full history.

Custom properties

Peripherals support custom fields (via `CustomPropertiesMixin`), just like hardware assets. Define fields in **Administration → Configuration → Custom Fields**.

Copyright © 2024–2026 pixelotes — Elastic License

2.4.3 Software Inventory

Track software applications deployed across your organization.

Registering software

1. Navigate to **Assets → Software**.
2. Click **Add Software**.
3. Provide: name, vendor, version, and license type.
4. Link to the assets where the software is installed.

Linking to assets

Software records can be linked to hardware assets to show what runs where. This feeds into the service catalog — when a business service depends on specific software, the full chain from service → software → asset is visible.

Version tracking

Update the version field when software is upgraded. The audit trail captures version changes with timestamps, providing a history of deployments.

Copyright © 2024–2026 pixelotes — Elastic License

2.4.4 Locations

Define physical and logical locations for asset tracking, compliance scoping, and organizational structure.

Location hierarchy

Locations support a parent-child hierarchy: Country → City → Building → Floor → Room. The tree view (powered by the `treeview.py` route) displays the full hierarchy with expandable nodes.

1. Navigate to **Administration → Locations**.
2. Click **Add Location**.
3. Provide: name, type, address, and optionally a parent location.

Asset assignment

Assets and peripherals can be assigned to locations. The asset list supports filtering by location, enabling quick inventory checks per site.

Copyright © 2024–2026 pixelotes — Elastic License

2.4.5 Maintenance

Log repairs, servicing, and preventive maintenance for hardware assets.

Creating maintenance logs

1. From an asset's detail page, click **Log Maintenance**.
2. Provide: issue description, resolution, cost, and time spent.
3. Set the maintenance type: corrective (fixing a problem) or preventive (scheduled servicing).
4. The asset status may change to **In Repair** during active maintenance.

Maintenance history

All maintenance logs are displayed on the asset's detail page in chronological order. This history is valuable for:

- Identifying assets with recurring issues (candidates for replacement).
- Tracking total cost of ownership (purchase price + cumulative maintenance costs).
- Demonstrating asset management practices for compliance audits.

Copyright © 2024–2026 pixelotes — Elastic License

2.4.6 Disposal

Manage end-of-life processes for hardware assets with method tracking and audit trail.

Disposal workflow

1. Change the asset status to **Awaiting Disposal**.
2. Navigate to **Assets → Disposals** or click **Create Disposal** from the asset detail page.
3. Record:
    - **Method** — e-waste recycling, resale, donation, destruction.
    - **Date** of disposal.
    - **Certificate** — upload disposal certificate from the vendor if applicable.
    - **Proceeds** — any financial return from resale.
4. Complete the disposal — the asset status changes to **Disposed**.

Audit trail

Disposed assets remain in the database with full history: procurement, assignments, maintenance, and disposal. This satisfies audit requirements for asset lifecycle documentation (ISO 27001 A.8).

The `DisposalHistory` model tracks status changes within the disposal workflow itself (from awaiting to completed).

Copyright © 2024–2026 pixelotes — Elastic License

2.5 Compliance & Audits

2.5.1 Frameworks & Controls

OpsDeck's compliance engine is built around frameworks — structured collections of controls that define your regulatory or security requirements.

Supported frameworks

OpsDeck ships with templates for:

- **ISO 27001:2022** — information security management controls.
- **SOC 2** — trust service criteria (security, availability, processing integrity, confidentiality, privacy).

You can also create **custom frameworks** for internal policies, industry regulations, or client-specific requirements.

Importing a framework

1. Navigate to **Compliance → Frameworks**.
2. Click **Add Framework**.
3. Choose from a template or create a blank framework.
4. For templates, controls are pre-populated with standard numbering and descriptions.
5. Customize as needed — add, remove, or modify controls.

Framework controls

Each control represents a single requirement:

- **Reference** — control number (e.g., A.8.1, CC6.1).
- **Title** — short description.
- **Description** — full requirement text.
- **Category/Section** — grouping within the framework.
- **Status** — compliance status (see Compliance Dashboard).

Mapping controls to evidence

Controls are connected to evidence through **Compliance Links** — the polymorphic linking system that connects any entity to any control. See Compliance Dashboard for details on evidence linking workflows.

Statement of Applicability (SoA)

For ISO 27001, the Statement of Applicability documents which controls are in scope and which are excluded with justification. OpsDeck supports this through:

- Marking controls as applicable or excluded.
- Adding justification notes for excluded controls.
- Generating an SoA report from the framework view.

Copyright © 2024–2026 pixelotes — Elastic License

2.5.2 Compliance Dashboard

The compliance dashboard provides a real-time view of your organization's compliance posture across all configured frameworks, with control-level status tracking and evidence coverage metrics.

Compliance score

The dashboard displays an overall compliance score per framework, calculated as the percentage of controls that have at least one evidence link (compliance link) attached. Controls are categorized by status:

Status	Meaning
Compliant	Control has sufficient evidence linked and is marked as satisfied
Manual	Control requires manual verification — evidence exists but needs review
Warning	Control has partial evidence or evidence is outdated
Non-compliant	Control has no evidence or has been explicitly marked as non-compliant
Uncovered	Control has not been addressed yet — no links, no status

Control status breakdown

Each framework page shows all controls grouped by category or section, with:

- Status badge per control.
- Count of linked evidence items.
- Last review date.
- Assigned owner (if set).

Click any control to see its detail page with the full list of compliance links — the assets, policies, services, suppliers, and activities linked as evidence.

Evidence linking

Evidence is linked to controls via the **Compliance Link** mechanism, which is available from two directions:

From the control. On the control detail page, click "Link Evidence" and search for the entity to link.

From any entity. On any asset, policy, service, or supplier detail page, scroll to "Compliance Links" and select the control to link to. Add context notes explaining how this entity satisfies the requirement.

> **Tip**
>
> Link evidence continuously as part of daily operations, not just during audit preparation. This builds a living evidence repository that makes audit snapshots comprehensive from day one.

Filtering by framework

If multiple frameworks are configured (e.g., ISO 27001 and SOC 2), the dashboard supports:

- Switching between frameworks via tabs.
- Cross-framework view showing controls that map to multiple standards.
- Filtering by status, category, or assigned owner.

Copyright © 2024–2026 pixelotes — Elastic License

Relationship to other modules

The compliance dashboard aggregates data from across OpsDeck:

- **Audit snapshots** — frozen compliance state at a point in time. See Audit Snapshots.
- **Compliance drift** — automatic detection of status regressions. See Compliance Drift.
- **UAR findings** — unresolved access findings can impact compliance status. See User Access Reviews.
- **Risk register** — risks can be linked to controls they threaten. See Risk Register.

Copyright © 2024–2026 pixelotes — Elastic License

2.5.3 Audit Snapshots

Audit snapshots capture the compliance state at a specific point in time, creating an immutable record for auditors. This is OpsDeck's "defense room" — the interface you present during an external audit.

Audit creation strategies

OpsDeck supports two approaches when creating an audit:

Strategy	When to use
Fresh start	First-time audit, new framework, or a complete reassessment
Renewal	Subsequent audits that build on a previous audit's scope and evidence

Renewal mode clones the previous audit's control items and evidence links, giving you a head start rather than rebuilding from scratch.

Creating an audit

1. Navigate to **Compliance → Audits**.
2. Click **Create Audit**.
3. Choose the target framework.
4. Select **Fresh Start** or **Renewal** (and pick the source audit if renewing).
5. The system creates `AuditControlItem` records — frozen copies of each framework control with their current compliance status and linked evidence.

Working with the audit

Once created, the audit provides:

- **Control list** — all controls in scope, grouped by framework section, with status badges.
- **Evidence links** — each control shows its linked evidence items (assets, policies, services, etc.) as they were at snapshot time.
- **Gap analysis** — controls without evidence are highlighted. Use this to identify gaps before the auditor reviews.
- **Notes** — add auditor notes, remediation plans, or context to any control item.

Linking additional evidence

While the audit is unlocked, you can:

1. Navigate to a control item within the audit.
2. Click **Link Evidence**.
3. Search for and select additional entities to link.
4. Each new link is recorded as an `AuditControlLink` within the audit scope.

Locking the audit

When the audit is complete:

1. Click **Lock Audit**.
2. This sets the audit to an immutable state — no further changes can be made.
3. The locked audit serves as a point-in-time record for auditors and regulators.

Copyright © 2024–2026 pixelotes — Elastic License

> **Warning**
>
> Locking is irreversible. Ensure all evidence is linked and all gaps are documented before locking.

Exporting for auditors

The audit export service generates a comprehensive evidence package:

- PDF report with all controls, their status, and linked evidence.
- Attachments referenced by evidence links.
- Summary statistics (compliant, non-compliant, gap counts).

Navigate to the audit detail page and click **Export** to generate the package.

Audit cloning

For recurring audits (e.g., annual ISO 27001 surveillance), cloning saves significant effort:

1. Create a new audit with **Renewal** strategy.
2. Select the previous audit as the source.
3. All control items and evidence links are copied to the new audit.
4. Review the cloned data — update statuses, add new evidence, remove obsolete links.
5. Lock when ready.

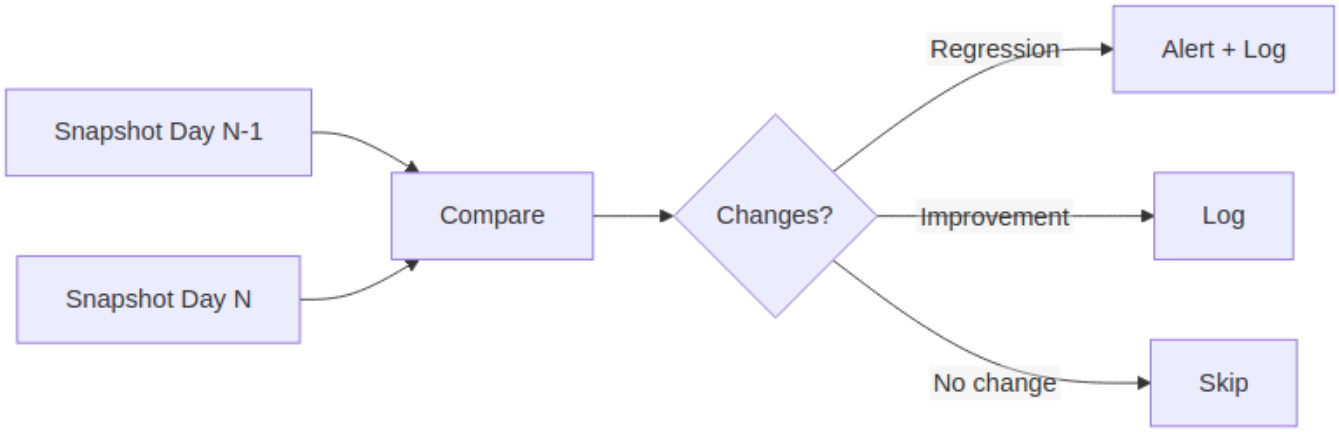
Copyright © 2024–2026 pixelotes — Elastic License

2.5.4 Compliance Drift

Compliance drift detection monitors your compliance posture over time and alerts when controls regress — catching problems before your next audit finds them.

How drift detection works

OpsDeck takes daily automated snapshots of compliance status across all frameworks. Each snapshot captures the status of every control at that point in time. The system then compares consecutive snapshots to detect changes.



Change classification

Classification	Description	Severity
Regression	Control status worsened	Critical (→ Non-compliant), High (→ Warning), Medium (other)
Improvement	Control status improved	Informational

Drift timeline

The drift dashboard displays a timeline visualization showing changes over configurable periods (7, 30, or 90 days). The timeline includes:

- **Statistics cards** — total regressions, improvements, and net change.
- **Event markers** — each day with detected changes is marked on the timeline.
- **Event details** — click any marker to see the control-by-control breakdown for that day.

Manual snapshots

In addition to the daily automated snapshot, you can capture a snapshot manually:

1. Navigate to **Compliance → Compliance Drift**.
2. Click **Capture Snapshot**.
3. The snapshot is compared immediately against the previous one.

This is useful before and after major changes (e.g., deploying a new policy, completing a remediation).

Copyright © 2024–2026 pixelotes — Elastic License

Alert configuration

Drift alerts are triggered when critical regressions are detected (any control moving to Non-compliant status):

- Logged to application logs for monitoring integration.
- Sent via configured notification channels (email, Slack webhook).
- Visible on the compliance drift dashboard with severity badges.

Resolving drift

When a regression is detected:

1. Click the drift event to see which controls regressed.
2. Investigate the cause — was evidence removed? Was a linked entity archived?
3. Address the root cause: re-link evidence, update the control status, or create a remediation task.
4. The next snapshot will capture the improvement.

> **Tip**
>
> Drift detection is most valuable when evidence is linked continuously. If you only link evidence during audit preparation, the drift timeline will show a "cliff" before each audit and won't catch regressions between audits.

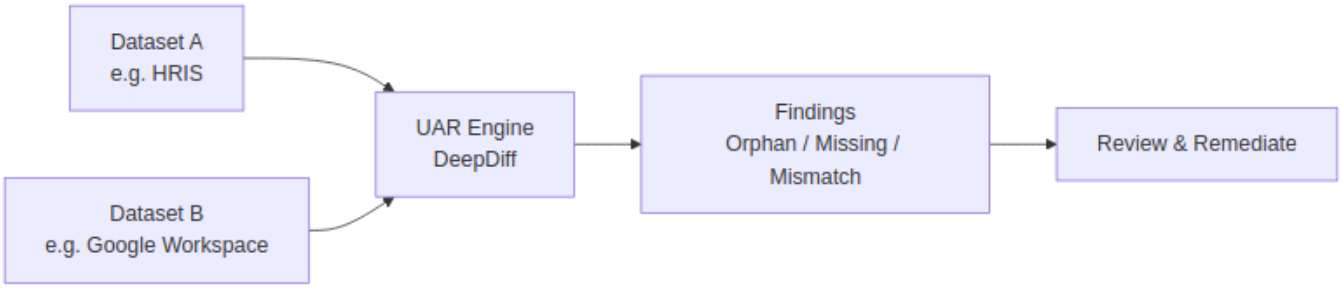
Copyright © 2024–2026 pixelotes — Elastic License

2.6 User Access Reviews

The UAR module automates the comparison of user records between systems to detect orphaned accounts, missing access, and mismatched attributes. It replaces manual spreadsheet-based access reviews with scheduled, repeatable comparisons.

2.6.1 How it works

A UAR comparison defines two datasets (e.g., HRIS vs. Identity Provider) and the fields to compare. When executed, the UAR engine loads both datasets, matches records by a key field (typically email or employee ID), and generates findings for any discrepancies.



2.6.2 Finding types

Type	Description	Severity
Orphan	Record exists in system A but not in system B. Example: employee in HRIS but no Google Workspace account.	Medium
Missing	Record exists in system B but not in system A. Example: Google account with no matching HRIS record (potential orphaned account).	High
Mismatch	Record exists in both systems but field values differ. Example: different department or role.	Low–Medium

2.6.3 Configuring a comparison

1. Navigate to **Compliance → User Access Reviews**.
2. Click **Create Comparison**.
3. Configure:
   - **Name** — descriptive label (e.g., "HRIS vs Google Workspace Q1 2026").
   - **Dataset A** — source system and data source (CSV upload, API, database query).
   - **Dataset B** — target system and data source.
   - **Key field** — the field used to match records (e.g., `email`).
   - **Comparison fields** — which fields to compare for mismatches (e.g., `department`, `role`, `status`).
4. Save.

2.6.4 Scheduling automatic execution

Comparisons can be scheduled to run automatically:

1. On the comparison detail page, click **Schedule**.
2. Choose frequency: daily, weekly, or monthly.
3. Select the time of day (UTC).
4. APScheduler triggers the execution automatically.

Copyright © 2024–2026 pixelotes — Elastic License

Each execution creates a `UARExecution` record with its own set of findings.

2.6.5 Reviewing findings

1. Navigate to the execution detail page.
2. Findings are listed by severity with counts per type.
3. For **mismatch** findings, click **View Details** to open the visual diff viewer — a side-by-side comparison highlighting field-by-field differences.
4. For each finding, you can:
    - **Resolve** — mark as addressed.
    - **Mark as false positive** — the finding is expected and not actionable.
    - **Create incident** — escalate to the security incident workflow.
    - **Add comment** — document the resolution or reasoning.

2.6.6 Bulk remediation

From the findings list:

- Select multiple findings using checkboxes.
- Apply bulk actions: resolve, assign, mark as false positive, export to CSV.
- Bulk operations log each action individually in the audit trail.

2.6.7 Compliance integration

UAR findings serve as evidence for access governance controls:

- Link UAR executions to compliance controls (e.g., SOC 2 CC6.1, ISO 27001 A.9.2).
- Regular execution provides ongoing evidence that access reviews are performed.
- Findings feed into compliance status — unresolved critical findings may trigger compliance drift alerts.

Copyright © 2024–2026 pixelotes — Elastic License

2.7 Risk Management

2.7.1 Risk Register

The risk register tracks identified threats to your organization, their scoring, treatment plans, and evolution over time.

Creating a risk

1. Navigate to **Security → Risks**.
2. Click **Add Risk**.
3. Fill in:
    - **Title** and **description** of the threat.
    - **Category** — selected from the risk taxonomy (or create a custom category).
    - **CIA classification** — which pillar is threatened: Confidentiality, Integrity, Availability, or a combination.
    - **Owner** — the person responsible for managing this risk.
4. Score the risk (see below).
5. Link affected items — assets, services, suppliers, or compliance controls.

Risk scoring

Each risk is scored on two axes:

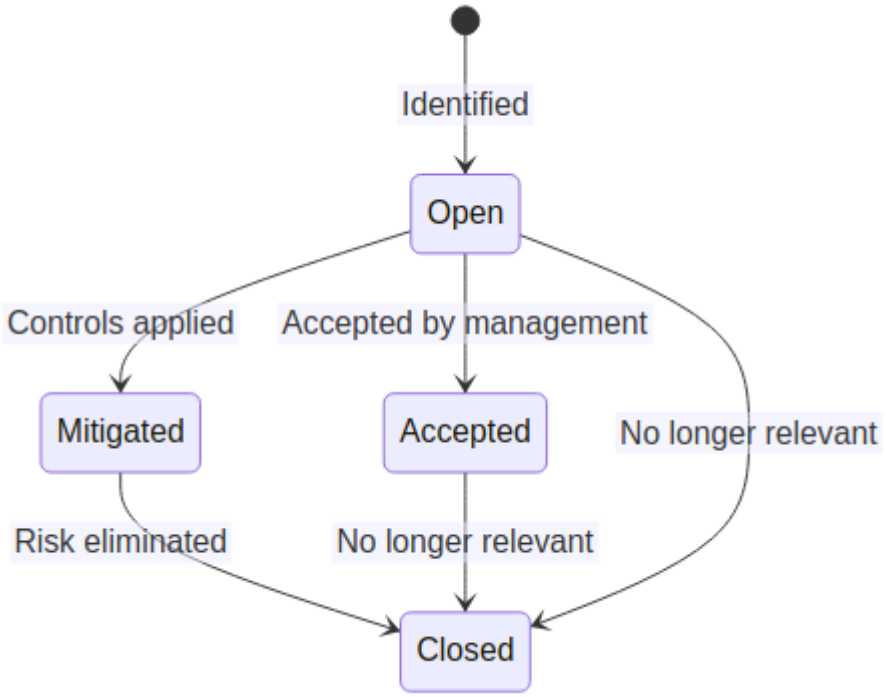
Dimension	Scale	Description
Impact	1–5	Consequence if the risk materializes
Likelihood	1–5	Probability of occurrence

Inherent risk = Impact × Likelihood (before any mitigation).

After documenting mitigating controls (linked security activities), set the **residual risk** — the remaining risk level after controls are applied.

Copyright © 2024–2026 pixelotes — Elastic License

Risk lifecycle



Risk history and trending

Every time a risk's score, status, or treatment plan changes, a `RiskHistory` record is created. The risk detail page shows a timeline of changes, enabling trending analysis across review cycles.

Risk catalog

OpsDeck includes a predefined risk catalog (`CatalogRisk`) with common IT risks organized by category. Use the catalog as a starting point when conducting risk assessments — select relevant risks and add them to your register with organization-specific scoring and context.

Linking risks to compliance

Risks can be linked to framework controls to show how identified threats map to compliance requirements. Conversely, compliance gaps can generate new risk entries. This bidirectional linking ensures that risk management and compliance management reinforce each other.

Copyright © 2024–2026 pixelotes — Elastic License

2.7.2 Risk Assessment

Risk assessments provide a structured workflow for evaluating threats across a defined scope, producing documented findings with evidence.

Assessment workflow

1. Navigate to **Security → Risk Assessments**.
2. Click **Create Assessment**.
3. Define the scope: which assets, services, or business areas are being assessed.
4. For each identified risk, create a `RiskAssessmentItem`:
    - Describe the threat.
    - Score impact and likelihood.
    - Document existing controls.
    - Attach evidence (screenshots, reports, configurations).
5. Set a **validity period** — the date range during which this assessment is considered current.
6. Complete the assessment to finalize findings.

Evidence attachment

Each assessment item supports evidence attachments:

- Upload files (PDFs, screenshots, configuration exports).
- Link to existing OpsDeck entities (assets, policies, compliance controls).
- Evidence is stored as `RiskAssessmentEvidence` records.

Assessment reports

Generate a summary report from the assessment detail page, including:

- Scope and methodology.
- Findings with scoring and evidence references.
- Recommended treatment plans.
- Risk heat map (impact vs. likelihood matrix).

Relationship to risk register

Findings from a risk assessment can be promoted to the risk register:

- Click **Add to Risk Register** on any assessment item.
- A `Risk` record is created with the scoring and context from the assessment.
- The assessment item links back to the risk for traceability.

Copyright © 2024–2026 pixelotes — Elastic License

2.7.3 Security Incidents

The incident management module tracks security events from initial report through investigation, resolution, and post-incident review.

Reporting an incident

1. Navigate to **Security → Incidents**.
2. Click **Report Incident**.
3. Provide:
    - **Title** and **description** — what happened.
    - **Severity** — Critical, High, Medium, Low.
    - **Affected systems** — link assets, services, or users impacted.
    - **Assignee** — the person responsible for investigation.

Incident timeline

Every incident has a timeline (`IncidentTimelineEvent`) that tracks the investigation:

- Add timeline entries as the investigation progresses.
- Each entry records the action taken, who took it, and when.
- The timeline provides a chronological audit trail of the response.

Post-incident review

After resolution:

1. Click **Create Post-Incident Review** on the incident detail page.
2. Document root cause, contributing factors, and lessons learned.
3. Link follow-up actions: new risks, policy updates, configuration changes.
4. The `PostIncidentReview` record is permanently linked to the incident.

Linking incidents

Incidents can be linked to:

- **Risks** — an incident may validate a previously identified risk or create a new one.
- **Compliance controls** — incident response demonstrates control effectiveness.
- **Assets and services** — affected infrastructure.
- **UAR findings** — an access-related incident may originate from a UAR finding.

Copyright © 2024–2026 pixelotes — Elastic License

2.8 Vendors & Procurement

2.8.1 Suppliers

The supplier management module tracks your vendors, their compliance status, contacts, and linked contracts and subscriptions.

Adding a supplier

1. Navigate to **Vendors → Suppliers**.
2. Click **Add Supplier**.
3. Provide: company name, website, category, and description.
4. Set the **compliance status**: Approved, Pending Review, Rejected, or Not Assessed.
5. Add notes about the vendor relationship.

Supplier contacts

Each supplier can have multiple contacts:

1. From the supplier detail page, click **Add Contact**.
2. Provide name, email, phone, and role.
3. Contacts are used for communication tracking and contract management.

Compliance status tracking

Supplier compliance status reflects whether the vendor meets your security and regulatory requirements:

Status	Meaning
Approved	Vendor has passed security review
Pending Review	Review in progress or scheduled
Rejected	Vendor does not meet requirements
Not Assessed	No review has been conducted

Linking to compliance controls

Suppliers can be linked to framework controls as evidence of vendor management practices (e.g., ISO 27001 A.15 — Supplier relationships):

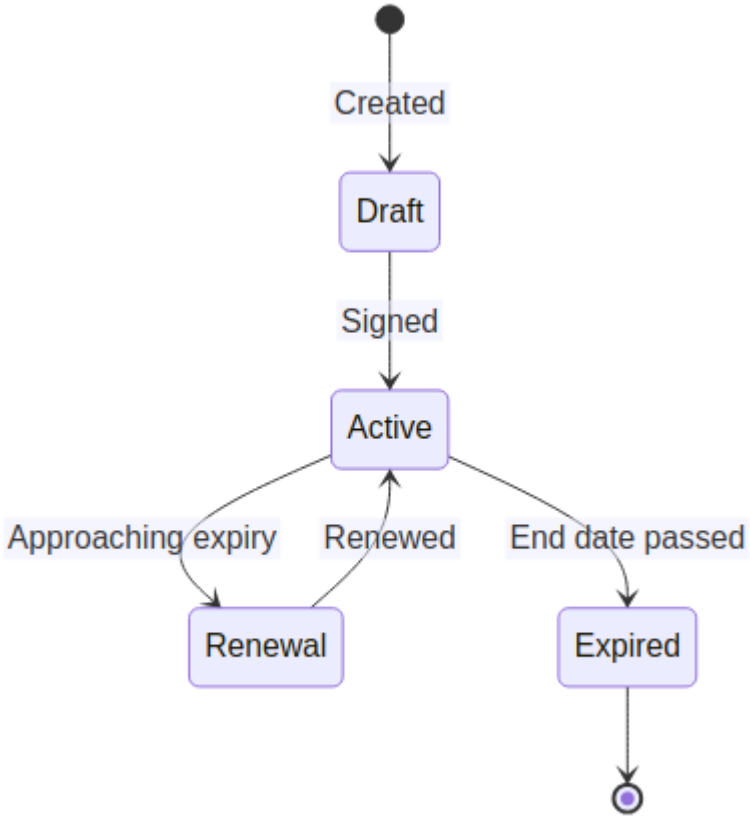
1. On the supplier detail page, scroll to **Compliance Links**.
2. Link to the relevant controls.
3. Add context notes about the supplier's role in satisfying the requirement.

Copyright © 2024–2026 pixelotes — Elastic License

2.8.2 Contracts

Track contract lifecycles with suppliers, including line items, renewal dates, and document attachments.

Contract lifecycle



Creating a contract

1. Navigate to **Vendors → Contracts**.
2. Click **Add Contract**.
3. Link to a **supplier**.
4. Set start date, end date, and value.
5. Add **contract items** (line items) with individual descriptions and costs.
6. Upload the signed contract document as an attachment.

Renewal tracking

- Contracts approaching their end date appear on the dashboard and in renewal reports.
- Notifications are sent before expiry based on configured lead times.
- Renewing a contract creates a new contract record linked to the original.

Copyright © 2024–2026 pixelotes — Elastic License

Document management

Attach supporting documents to any contract:

- Signed agreements, amendments, SLAs.
- Vendor security assessments.
- Pricing schedules.

Copyright © 2024–2026 pixelotes — Elastic License

2.8.3 Subscriptions

Track SaaS subscriptions with renewal forecasting, cost optimization insights, and access management links.

Subscription tracking

1. Navigate to **Vendors → Subscriptions**.
2. Click **Add Subscription**.
3. Provide: name, supplier, plan details, and cost.
4. Set **renewal date** and **billing cycle** (monthly, annual, etc.).
5. Link to a budget and cost center for financial tracking.

Renewal forecasting

OpsDeck generates a 12-month renewal forecast based on active subscriptions:

- View upcoming renewals on the dashboard.
- The finance service calculates projected costs per period.
- Renewals with auto-renew enabled are flagged separately.

Cost optimization

The subscription list view supports filtering and sorting by cost, enabling:

- Identification of unused or underutilized subscriptions.
- Comparison of overlapping tools.
- License count vs. active user tracking (when linked to user access data).

Access management

Link subscriptions to user access data:

- Track which users have access to each subscription.
- UAR comparisons can use subscription user lists as a dataset.
- When a subscription is cancelled, linked access records flag orphaned accounts.

Copyright © 2024–2026 pixelotes — Elastic License

2.8.4 Purchases & Budgets

Track purchase orders, manage budgets with validity periods, allocate costs to cost centers, and monitor spending.

Purchase orders

1. Navigate to **Vendors → Purchases**.
2. Click **Add Purchase**.
3. Link to a supplier and optionally a contract.
4. Set purchase date, amount, and payment method.
5. Assign to a budget and cost center.
6. Upload supporting documents (invoices, receipts).

Budget management

Budgets have defined validity periods and track spending:

1. Navigate to **Finance → Budgets**.
2. Click **Add Budget**.
3. Set the budget name, amount, start and end dates.
4. Purchases are validated against the budget's validity period — a purchase outside the valid dates triggers a warning.
5. The budget detail page shows allocated vs. remaining funds.

Cost centers

Cost centers represent departments or organizational units for financial allocation:

- Assets, purchases, and subscriptions can be assigned to cost centers.
- Reports show spending per cost center.

Payment methods

Track how purchases are paid:

- Credit cards, wire transfers, purchase orders.
- Link payment methods to suppliers or cost centers.
- Cost history (`CostHistory`) tracks price changes over time.

Copyright © 2024–2026 pixelotes — Elastic License

2.9 Service Catalog

The service catalog maps business services to their technical dependencies, enabling impact analysis, compliance tracking, and operational visibility.

2.9.1 Business services

A business service represents a capability delivered to the organization or its customers — "Customer Portal," "Payment Processing," "HR System," etc.

1. Navigate to **Services → Business Services**.
2. Click **Add Service**.
3. Provide: name, description, owner, and **criticality** (Critical / Non-Critical).

2.9.2 Service components

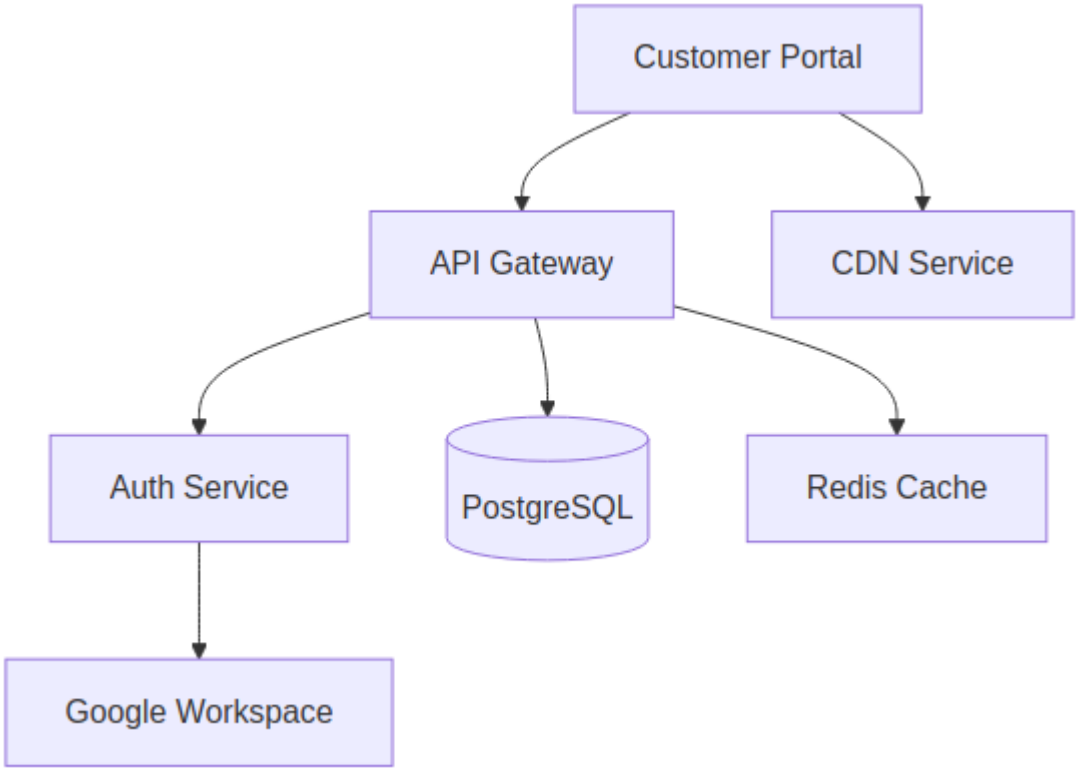
Components are the technical building blocks that support a business service:

1. From the service detail page, click **Add Component**.
2. Link:
    - **Assets** — servers, databases, network devices.
    - **Software** — applications running on those assets.
    - **Subscriptions** — SaaS services the business service depends on.
    - **Other services** — upstream or downstream service dependencies.

2.9.3 Dependency mapping

The topology view shows the complete dependency graph for a service:

Copyright © 2024–2026 pixelotes — Elastic License



The `dependency_graph.py` utility resolves transitive dependencies — if Service A depends on Service B which depends on Database C, the graph shows all three levels.

2.9.4 Impact analysis

When an incident or change affects a component:

- Navigate to the affected asset or service.
- The **Impact** section shows all business services that depend on it.
- This informs incident severity assessment and change approval decisions.

2.9.5 Compliance context

Services can be linked to:

- **Policies** that govern the service.
- **Compliance controls** the service must satisfy.
- **Security activities** that protect the service.
- **Risks** that threaten the service.

This provides a unified view of a service's compliance posture.

Copyright © 2024–2026 pixelotes — Elastic License

2.10 People

2.10.1 Users & Groups

Manage user accounts, group memberships, role assignments, and the organization chart.

User management

CREATING USERS

1. Navigate to **People → Users**.
2. Click **Add User**.
3. Provide: name, email, department, location, and role.
4. The user receives a temporary password (or uses Google OAuth if configured).
5. First login requires a password change.

ROLES

Role	Description
Admin	Full system access including user management and configuration
Manager	Read/write access to operational data, limited admin capabilities
User	Read access to assigned data with limited write permissions

CUSTOM PROPERTIES

Users support custom fields (via `CustomPropertiesMixin`). Define custom fields in **Administration → Configuration → Custom Fields** and they appear on all user forms.

Groups

Groups provide permission inheritance — all members of a group receive the group's permissions:

1. Navigate to **People → Groups**.
2. Click **Add Group**.
3. Name the group and assign module permissions.
4. Add users as members.

When a user belongs to multiple groups, permissions are merged — the highest access level wins (WRITE > READ_ONLY).

Organization chart

OpsDeck includes an interactive org chart:

- Navigate to **People → Organization** to view the hierarchy.
- The chart is rendered using OrgChart.js.
- `OrgChartSnapshot` records capture point-in-time snapshots of the organizational structure.

Permission assignment

See Permissions & RBAC for the technical details of how permissions are resolved and cached.

Copyright © 2024–2026 pixelotes — Elastic License

2.10.2 Onboarding & Offboarding

Manage employee onboarding and offboarding with templated checklists, pack communications, and progress tracking.

Onboarding packs

An onboarding pack defines the standard set of items needed for a new hire:

1. Navigate to **People → Onboarding**.
2. Click **Create Pack**.
3. Add **pack items** — tasks to complete (provision laptop, create accounts, assign training, etc.).
4. Each item can be assigned to a responsible person.

Process templates

Templates allow reuse across multiple onboarding processes:

1. Create a `ProcessTemplate` with standard items.
2. When onboarding a new hire, instantiate the template to create a `ProcessItem` checklist.
3. Items are checked off as they're completed, with timestamp and responsible user recorded.

Offboarding

The offboarding workflow mirrors onboarding in reverse:

1. Create an `OffboardingProcess` for the departing user.
2. Checklist items include: revoke access, collect hardware, update asset assignments, remove from groups.
3. Track completion to ensure nothing is missed.

Pack communications

Onboarding packs can trigger communications:

- Welcome emails with account details.
- Scheduled reminders for incomplete items.
- Manager notifications when the process is complete.

Copyright © 2024–2026 pixelotes — Elastic License

2.10.3 Hiring Pipeline

Track candidates through customizable hiring stages.

Hiring stages

Define the stages of your hiring process: Application Received → Phone Screen → Technical Interview → Offer → Hired (or Rejected). Stages are configured per organization.

Candidate tracking

1. Navigate to **People → Hiring**.
2. Click **Add Candidate**.
3. Provide: name, email, position applied for, and current stage.
4. Move candidates between stages as the process progresses.
5. Each stage transition is logged with timestamp and notes.

When a candidate is hired, they can be converted to a user record, which triggers the onboarding workflow.

Copyright © 2024–2026 pixelotes — Elastic License

2.11 Policies & Training

2.11.1 Policy Management

Manage organizational policies with version control and acknowledgment tracking.

Creating policies

1. Navigate to **Compliance → Policies**.
2. Click **Add Policy**.
3. Provide: title, description, and effective date.
4. Upload the policy document (PDF).
5. Set a **review schedule** (e.g., annual review).
6. Assign to users or groups for acknowledgment.

Policy versions

Policies support versioning:

- When updating a policy, create a new `PolicyVersion` rather than overwriting.
- Previous versions are preserved for audit trail.
- Users who acknowledged a previous version may need to re-acknowledge the new version.

Acknowledgment workflows

1. Assign users or groups to a policy.
2. Users see pending acknowledgments on their dashboard.
3. The user opens the policy, reads it, and clicks **Acknowledge**.
4. A `PolicyAcknowledgement` record is created with user ID and timestamp.
5. The tracking report shows acknowledgment rates per policy.

> **Tip**
>
> Link policies to compliance controls to demonstrate that governance requirements are met and that staff have been informed.

Compliance integration

Policies are one of the most common evidence types linked to compliance controls. Examples:

- Information Security Policy → ISO 27001 A.5.1
- Acceptable Use Policy → SOC 2 CC1.1
- Data Classification Policy → ISO 27001 A.8.2

Copyright © 2024–2026 pixelotes — Elastic License

2.11.2 Training Courses

Track security awareness training and compliance-related courses with assignment and completion workflows.

Course management

1. Navigate to **People → Training**.
2. Click **Add Course**.
3. Provide: title, description, content type (internal/external), and duration.
4. Set a completion deadline if required.

Assignment and completion

ASSIGNING TRAINING

1. From the course detail page, click **Assign**.
2. Select individual users or entire groups.
3. Set a due date.
4. Users are notified of the assignment.

COMPLETING TRAINING

1. Users see their assigned courses on their profile or dashboard.
2. After completing the training, the user (or a manager) marks it as complete.
3. Upload completion evidence (certificates, test results) if required.
4. A `CourseCompletion` record is created.

Compliance linking

Training records can be linked to compliance controls to demonstrate awareness requirements:

- ISO 27001 A.7.2.2 — Information security awareness, education and training.
- SOC 2 CC1.4 — The entity demonstrates a commitment to attract, develop, and retain competent individuals.

Link completed training campaigns to the relevant controls as evidence.

Copyright © 2024–2026 pixelotes — Elastic License

2.12 Credentials Vault

Store and manage sensitive credentials (API keys, service account passwords, shared secrets) with encryption at rest and access control.

2.12.1 Storing credentials

1. Navigate to **Security → Credentials**.
2. Click **Add Credential**.
3. Provide: name, description, category, and the secret value.
4. The secret is encrypted using Fernet symmetric encryption (from the `cryptography` library) before storage.
5. Optionally set a **rotation date** to track when the credential should be changed.

2.12.2 Access control

- Viewing decrypted secrets requires explicit permission (`can_read` on the credentials module).
- The credential list shows metadata (name, category, rotation date) without revealing secrets.
- Clicking **Reveal** decrypts and displays the secret temporarily.
- All reveal actions are logged in the audit trail.

2.12.3 Credential rotation tracking

- Set a rotation date when creating or updating a credential.
- The dashboard and reports show credentials approaching or past their rotation date.
- After rotating a credential in the external system, update the stored value and reset the rotation date.

> **Warning**
>
> The encryption key is derived from your `SECRET_KEY`. If you change `SECRET_KEY`, all stored credentials become unreadable. Always back up your `SECRET_KEY` securely.

Copyright © 2024–2026 pixelotes — Elastic License

2.13 Certificates

Track TLS/SSL certificates and other digital certificates with expiry alerting and version history.

2.13.1 Certificate tracking

1. Navigate to **Security → Certificates**.
2. Click **Add Certificate**.
3. Provide: name, domain/subject, issuer, and expiry date.
4. Link to the services or assets that use this certificate.

2.13.2 Expiry alerts

- The scheduler checks daily for certificates approaching expiry.
- Notifications are sent based on configured lead times (e.g., 30, 14, 7 days before expiry).
- Expired certificates are flagged on the dashboard.

2.13.3 Certificate versions

When renewing a certificate:

1. Create a new `CertificateVersion` on the existing certificate record.
2. Update the expiry date and any changed details (issuer, key type).
3. Previous versions are preserved in the history.

2.13.4 Linking to infrastructure

Link certificates to:

- **Business services** that use them (customer portal, API gateway).
- **Assets** that host them (load balancers, web servers).
- **Compliance controls** related to encryption requirements.

Copyright © 2024–2026 pixelotes — Elastic License

2.14 Change Management

Track infrastructure and configuration changes with categorization and impact assessment.

2.14.1 Change request workflow

1. Navigate to **Operations → Changes**.
2. Click **Add Change**.
3. Provide:
    - **Title** and **description** of the change.
    - **Category** — Standard, Normal, or Emergency.
    - **Impact** and **urgency** assessment.
    - **Planned date** and implementation window.
4. Link affected assets and services.
5. Submit for approval.

2.14.2 Change types

Category	Description
Standard	Pre-approved, low-risk, routine changes
Normal	Requires assessment and approval before implementation
Emergency	Urgent changes that bypass normal approval (documented post-hoc)

2.14.3 Change lifecycle

Changes move through statuses from draft through implementation to completion. Each status transition is recorded in the audit trail. Post-implementation, the change record documents whether the outcome matched expectations.

Copyright © 2024–2026 pixelotes — Elastic License

2.15 Business Continuity & Disaster Recovery

Document BCDR plans and track test execution to demonstrate resilience readiness.

2.15.1 BCDR plans

1. Navigate to **Security → BCDR**.
2. Click **Add Plan**.
3. Document:
   - **Plan scope** — which services and systems are covered.
   - **Recovery objectives** — RTO (Recovery Time Objective) and RPO (Recovery Point Objective).
   - **Procedures** — step-by-step recovery instructions.
   - **Responsible parties** — who executes the plan.
4. Link to the business services covered by the plan.

2.15.2 Test log tracking

Regular BCDR testing is required by most compliance frameworks:

1. From the plan detail page, click **Log Test**.
2. Record: test date, test type (tabletop, simulation, full failover), and results.
3. Document lessons learned and any plan updates needed.
4. `BCDRTestLog` records provide evidence for compliance controls.

2.15.3 Linking to compliance

BCDR plans and test logs are commonly linked to:

- ISO 27001 A.17 — Information security aspects of business continuity management.
- SOC 2 A1 — Additional criteria for availability.

Copyright © 2024–2026 pixelotes — Elastic License

2.16 CRM

2.16.1 Contacts & Leads

The CRM module tracks external contacts, leads, and requirements for business development and partner management.

Contact management

Contacts represent people at external organizations (suppliers, partners, prospects):

1. Navigate to **CRM → Contacts**.
2. Click **Add Contact**.
3. Provide: name, email, phone, organization, and role.
4. Link contacts to suppliers for vendor management integration.

Lead tracking

Leads represent potential business opportunities in early stages:

1. Navigate to **CRM → Leads**.
2. Click **Add Lead**.
3. Provide: company name, contact details, source, and status.
4. Convert qualified leads to opportunities.

Requirements and actions

Track requirements from contacts (feature requests, compliance demands, integration needs):

- Create `Requirement` records linked to contacts.
- Add `RequirementAction` items for follow-up tasks.
- Track status from received through assessment to completion.

Copyright © 2024–2026 pixelotes — Elastic License

2.16.2 Opportunities

Track sales and partnership opportunities through the pipeline with tasks and revenue projections.

Opportunity pipeline

1. Navigate to **CRM → Opportunities**.
2. Click **Add Opportunity**.
3. Provide: title, contact, expected value, probability, and stage.
4. Move opportunities through stages as they progress.

Tasks and activities

Each opportunity supports:

- **Tasks** (`OpportunityTask`) — action items with due dates and assignees.
- **Activities** (`Activity`) — logged interactions (calls, meetings, emails) with notes and outcomes.

Revenue tracking

Set expected value and close probability per opportunity. The CRM dashboard aggregates pipeline value by stage and projected close date.

Copyright © 2024–2026 pixelotes — Elastic License

2.16.3 Campaigns

Manage email campaigns for announcements, policy communications, and awareness initiatives.

Email campaigns

1. Navigate to **Communications → Campaigns**.
2. Click **Create Campaign**.
3. Select or create an **email template** (`EmailTemplate`).
4. Define the recipient list (users, groups, or custom lists).
5. Schedule or send immediately.

Templates

Email templates use a customizable format with merge fields for personalization (name, email, role). Create templates in **Communications → Templates**.

Scheduled communications

`ScheduledCommunication` records allow scheduling campaigns for future delivery. The APScheduler processes the queue and sends emails via the configured SMTP server. Delivery status is tracked per recipient.

Copyright © 2024–2026 pixelotes — Elastic License

2.17 Reports & Search

OpsDeck includes built-in reports and a universal search engine for cross-entity discovery.

2.17.1 Built-in reports

Available from **Reports** in the sidebar:

Report	Content
Asset inventory	Complete asset list with status, location, assignment, and warranty
Compliance summary	Per-framework control status and evidence coverage
Subscription forecast	12-month renewal projection with cost breakdown
UAR findings	Open findings across all comparisons
Risk register	All risks with scoring, treatment, and status
Certificate status	Certificates by expiry date
Training compliance	Course completion rates per user and group

Reports can be exported as CSV or PDF (via WeasyPrint).

2.17.2 Universal search

Access via the search icon in the top navigation bar:

1. Enter a query — searches across names, descriptions, serial numbers, notes, and custom field values.
2. Results are grouped by entity type (assets, users, incidents, suppliers, etc.) with result counts.
3. Apply **faceted filters**: entity type, status, date range, severity.
4. Click any result to navigate to its detail page.

The search service (`search_service.py`) queries all configured entity types in parallel and merges results.

2.17.3 Export options

Most list views and reports support:

- **CSV export** — all columns, respecting current filters.
- **PDF export** — formatted report using WeasyPrint with headers, pagination, and summary statistics.

Copyright © 2024–2026 pixelotes — Elastic License

2.18 Configuration

Manage organization-wide settings, custom fields, notification preferences, and taxonomy.

2.18.1 Organization settings

Navigate to **Administration → Configuration**:

- Organization name and logo.
- Default timezone and date format.
- Currency settings for financial modules.
- Notification preferences (global email and webhook settings).

Settings are stored in the `OrganizationSettings` model and cached for performance.

2.18.2 Custom fields

Define additional fields for assets, users, and peripherals:

1. Navigate to **Administration → Configuration → Custom Fields**.
2. Click **Add Custom Field**.
3. Provide: field name, data type (text, number, date, select), and target entity types.
4. The field appears on all forms and detail pages for the selected entity types.
5. Custom field values are searchable via universal search.

2.18.3 Tags

Tags provide flexible, cross-entity categorization:

- Create tags in **Administration → Tags**.
- Apply tags to any entity that supports them.
- Filter lists and reports by tag.

2.18.4 Notification settings

Configure per-event notification preferences:

- Which events trigger notifications (asset assignment, incident creation, renewal approaching).
- Notification channels: email (via SMTP) and/or webhook (Slack, Discord).
- Per-user notification preferences can override global settings.

2.18.5 Configuration versioning

The `ConfigurationVersion` model tracks changes to system configuration over time, providing an audit trail of who changed what settings and when.

Copyright © 2024–2026 pixelotes — Elastic License

2.19 Data Import (CLI)

OpsDeck provides CLI commands for bulk CSV import, useful for initial system bootstrapping or migrating from legacy tools.

2.19.1 General usage

All import commands use the Flask CLI. For Docker deployments:

```
# Copy CSV into the container
docker cp my_data.csv opsdeck_web_1:/app/

# Run the import
docker exec -it opsdeck_web_1 flask data-import [COMMAND] my_data.csv
```

For local installations, run directly:

```
flask data-import [COMMAND] my_data.csv
```

2.19.2 Supported entities

Users

```
flask data-import users users.csv
```

Column	Required	Description
`name`	Yes	Full name
`email`	Yes	Email (must be unique)

Imported users receive the `user` role by default. Random passwords are generated and displayed in console output.

Assets

```
flask data-import assets assets.csv
```

Column	Required	Description
`name`	Yes	Asset name
`model`	No	Model identifier
`serial_number`	No	Serial number
`asset_type`	No	Type (laptop, server, etc.)
`status`	No	Lifecycle status

Suppliers

```
flask data-import suppliers suppliers.csv
```

Column	Required	Description
`name`	Yes	Company name
`website`	No	URL
`category`	No	Supplier category

Copyright © 2024–2026 pixelotes — Elastic License

Contacts

```
flask data-import contacts contacts.csv
```

Column	Required	Description
`name`	Yes	Contact name
`email`	Yes	Email
`supplier`	Yes	Supplier name (must exist)
`role`	No	Role at the supplier

2.19.3 Validation and error handling

- Duplicate records (matched by email for users, name for suppliers) are skipped with a warning.
- Invalid rows are logged with the row number and error details.
- The import runs within a database transaction — if critical errors occur, the entire import can be rolled back.
- Console output shows a summary: imported, skipped, and failed counts.

Copyright © 2024–2026 pixelotes — Elastic License

2.20 REST API

OpsDeck provides a REST API for programmatic access to all major entities, built with flask-smorest and documented via OpenAPI/Swagger.

2.20.1 Authentication

All API requests require a bearer token in the `Authorization` header:

```
Authorization: Bearer <your-api-token>
```

Generating a token

1. Navigate to your **User Profile**.
2. Scroll to **Developer Settings (API)**.
3. Click **Generate New Token**.
4. Copy the token immediately — it's displayed only once.

The token inherits the permissions of the owning user. Tokens are hashed (SHA-256) before storage and can be revoked from the user profile.

2.20.2 Swagger UI

Interactive API documentation is available at:

```
https://your-opsdeck-instance/swagger-ui
```

The Swagger UI lets you browse endpoints, see request/response schemas, and test API calls directly from the browser (with a valid token).

2.20.3 Supported endpoints

The API covers the following entities under `/api/v1/`:

Endpoint	Methods	Entity
`/api/v1/users`	GET	Users
`/api/v1/assets`	GET	Hardware assets
`/api/v1/peripherals`	GET	Peripherals
`/api/v1/licenses`	GET	Software licenses
`/api/v1/subscriptions`	GET	SaaS subscriptions
`/api/v1/services`	GET	Business services
`/api/v1/changes`	GET, POST	Change records
`/api/v1/incidents`	GET, POST	Security incidents
`/api/v1/onboarding`	GET, POST	Onboarding processes

Read-only entities (users, assets, etc.) support GET with pagination and filtering. Writable entities (changes, incidents, onboarding) support both GET and POST.

Copyright © 2024–2026 pixelotes — Elastic License

2.20.4 Pagination and filtering

List endpoints support:

```
GET /api/v1/assets?page=1&page_size=25
```

Response includes pagination metadata:

```
{
  "items": [...],
  "total": 142,
  "page": 1,
  "page_size": 25,
  "pages": 6
}
```

2.20.5 Example: list all assets

```
curl -H "Authorization: Bearer YOUR_TOKEN" \
  https://opsdeck.example.com/api/v1/assets
```

2.20.6 Example: create an incident

```
curl -X POST \
  -H "Authorization: Bearer YOUR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"title": "Unauthorized access attempt", "severity": "High", "description": "..."}' \
  https://opsdeck.example.com/api/v1/incidents
```


Copyright © 2024–2026 pixelotes — Elastic License

3. Deployment

3.1 Deployment Guide

This section covers how to install, configure, and operate OpsDeck in production environments.

3.1.1 Deployment options

Method	Best for	Complexity
Docker Compose	Small teams, single-server deployments	Low
Kubernetes (Helm)	Scalable, multi-node, cloud-native environments	Medium
Manual	Air-gapped environments, custom setups	High

3.1.2 Quick path

1. Start with Quick Start for a local dev instance
2. Review Environment Variables for configuration
3. Pick your deployment method above
4. Set up Backup & Restore
5. Review Security Hardening before going live

3.1.3 Requirements

- **Python** 3.11+
- **PostgreSQL** 15+ (16 recommended)
- **Storage** for attachments (local filesystem or S3-compatible)

Copyright © 2024–2026 pixelotes — Elastic License

3.2 Quick Start

Get a local development instance running in under 5 minutes.

3.2.1 Prerequisites

- Python 3.11+
- PostgreSQL 15+ running locally (or use the Docker Compose method below)
- Git

3.2.2 Option A: Local Python

```
# Clone
git clone https://github.com/pixelotes/opsdeck.git
cd opsdeck

# Virtual environment
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Environment
cp .env.example .env
# Edit .env and set DATABASE_URL to your local PostgreSQL

# Database
flask db upgrade
flask init-db
flask seed-db-prod    # Creates admin user and base data

# Run
flask run --debug
```

3.2.3 Option B: Docker Compose

```
git clone https://github.com/pixelotes/opsdeck.git
cd opsdeck
docker-compose up -d --build
```

This starts both the application and a PostgreSQL 16 container. The database is initialized automatically via `entrypoint.sh`.

3.2.4 First login

Open `http://127.0.0.1:5000` and log in with:

Field	Value
Email	`admin@example.com`
Password	`admin123`

You'll be prompted to change the password immediately.

> **Tip**
>
> Customize the initial admin credentials by setting `DEFAULT_ADMIN_EMAIL` and `DEFAULT_ADMIN_INITIAL_PASSWORD` in `.env` before the first run.

3.2.5 What's next

- Environment Variables — full configuration reference.
- Docker Compose — production-grade Docker deployment.

Copyright © 2024–2026 pixelotes — Elastic License

- Kubernetes — Helm chart deployment.
- Getting Started (User Guide) — learn the application.

Copyright © 2024–2026 pixelotes — Elastic License

3.3 Docker Compose Deployment

Production-grade deployment using Docker Compose with PostgreSQL, persistent storage, and health checks.

3.3.1 Prerequisites

- Docker Engine 24+
- Docker Compose v2

3.3.2 docker-compose.yml

The repository includes a `docker-compose.yml` configured for development. For production, use the following as a starting point:

```yaml
services:
  web:
    build:
      context: .
    ports:
      - "5000:5000"
    volumes:
      - ./data/attachments:/app/data/attachments
    environment:
      - DATABASE_URL=postgresql://opsdeck:${DB_PASSWORD}@db:5432/opsdeck
      - SECRET_KEY=${SECRET_KEY}
      - FLASK_DEBUG=0
      - DEFAULT_ADMIN_EMAIL=${ADMIN_EMAIL:-admin@example.com}
      - DEFAULT_ADMIN_INITIAL_PASSWORD=${ADMIN_PASSWORD:-admin123}
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: opsdeck
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: opsdeck
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U opsdeck -d opsdeck"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

3.3.3 Configuration

Create a `.env` file with production values:

```
SECRET_KEY=your-long-random-secret-key-here
DB_PASSWORD=strong-database-password
ADMIN_EMAIL=admin@yourcompany.com
ADMIN_PASSWORD=InitialSecurePassword123!
```

> **Danger**
>
> Never commit `.env` with production credentials to version control.

See Environment Variables for the full reference.

Copyright © 2024–2026 pixelotes — Elastic License

3.3.4 Build and run

```
docker-compose up -d --build
```

The `entrypoint.sh` script handles first-run initialization automatically:

1. Runs `flask db upgrade` to apply database migrations.
2. Runs `flask init-db` to create base tables and the admin user.
3. Starts Gunicorn with the configured number of workers.

Verify the deployment:

```
docker-compose ps
docker-compose logs -f web
```

3.3.5 Data persistence

Data	Container path	Host mount
Database	`/var/lib/postgresql/data`	`postgres_data` named volume
Attachments	`/app/data/attachments`	`./data/attachments` bind mount
Logs	`/app/logs`	Optional bind mount

3.3.6 TLS termination

Do not expose port 5000 directly to the internet. Place a reverse proxy in front:

Nginx **Traefik**

```
server {
    listen 443 ssl;
    server_name opsdeck.yourcompany.com;

    ssl_certificate /etc/letsencrypt/live/opsdeck.yourcompany.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/opsdeck.yourcompany.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Add labels to the `web` service in `docker-compose.yml`:

```
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.opsdeck.rule=Host(`opsdeck.yourcompany.com`)"
  - "traefik.http.routers.opsdeck.tls.certresolver=letsencrypt"
```

3.3.7 Network isolation

Isolate the database from external access:

```
networks:
  frontend:
  backend:

services:
  web:
    networks: [frontend, backend]
  db:
    networks: [backend]
```


Copyright © 2024–2026 pixelotes — Elastic License

3.3.8 Performance tuning

Gunicorn workers. The default formula is `(2 × CPU cores) + 1`. Set via the `WEB_CONCURRENCY` environment variable or by modifying `entrypoint.sh`.

Database connection pool. Configure in `.env` or application config:

```
SQLALCHEMY_POOL_SIZE=10
SQLALCHEMY_POOL_RECYCLE=3600
```

Container resource limits:

```yaml
deploy:
  resources:
    limits:
      cpus: '2'
      memory: 2G
    reservations:
      cpus: '1'
      memory: 1G
```

3.3.9 Backup

See Backup & Restore for automated backup scripts.

Quick manual backup:

```
# Database
docker-compose exec -T db pg_dump -U opsdeck opsdeck | gzip > backup_$(date +%Y%m%d).sql.gz

# Attachments
tar -czf attachments_$(date +%Y%m%d).tar.gz ./data/attachments
```

3.3.10 Troubleshooting

Container won't start. Check `docker-compose logs web`. Common causes: wrong `DATABASE_URL`, port 5000 already in use, missing `.env` file.

Database migration errors. Run `docker-compose exec web flask db upgrade` manually to see detailed errors.

Performance issues. Check `docker stats` for resource usage. Enable query logging with `SQLALCHEMY_ECHO=True` to find slow queries.

Copyright © 2024–2026 pixelotes — Elastic License

3.4 Kubernetes Deployment (Helm)

Deploy OpsDeck on Kubernetes using the included Helm chart, with support for internal or external PostgreSQL, persistent storage, and Ingress.

3.4.1 Prerequisites

- Kubernetes cluster v1.19+
- Helm 3.0+
- `kubectl` configured for your cluster

3.4.2 Chart structure

```
helm/
├── Chart.yaml          # Chart metadata and PostgreSQL subchart dependency
├── Chart.lock
├── values.yaml         # Default configuration
├── charts/             # Bundled PostgreSQL subchart
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    └── pvc.yaml
```

3.4.3 Installation

1. Create namespace and secrets

```
kubectl create namespace opsdeck

kubectl create secret generic opsdeck-secrets \
  --from-literal=secret-key='$(openssl rand -hex 32)' \
  --from-literal=admin-email='admin@yourcompany.com' \
  --from-literal=admin-password='SecurePassword123!' \
  -n opsdeck
```

2. Configure values

The key decision is database mode — **internal** (deploys a PostgreSQL pod) or **external** (uses an existing database like RDS/Aurora):

Internal PostgreSQL **External PostgreSQL (RDS/Aurora)**

```
# values.yaml
database:
  type: internal

postgresql:
  auth:
    username: opsdeck
    password: opsdeck-db-password
    database: opsdeck

# values.yaml
database:
  type: external
  external:
    host: "my-aurora-db.aws.com"
    port: 5432
    username: "opsdeck"
    database: "opsdeck"
    existingSecret:
      name: "opsdeck-db-secret"
      key: "password"
```

3. Configure persistent storage

```
persistence:
  logs:
```


Copyright © 2024–2026 pixelotes — Elastic License

```
    enabled: true
    size: 500Mi
    storageClass: ""       # Uses cluster default
    existingClaim: ""      # Or reference an existing PVC

  attachments:
    enabled: true
    size: 5Gi
    storageClass: ""
    existingClaim: ""
```

4. Install

```
helm upgrade --install opsdeck ./helm \
  --namespace opsdeck \
  --set image.tag=latest
```

5. Verify

```
kubectl get pods -n opsdeck
kubectl logs -f deployment/opsdeck -n opsdeck
```

3.4.4 Ingress

Enable and configure Ingress in `values.yaml`:

```
ingress:
  enabled: true
  className: nginx
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
  hosts:
    - host: opsdeck.yourcompany.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: opsdeck-tls
      hosts:
        - opsdeck.yourcompany.com
```

3.4.5 ArgoCD deployment

For GitOps workflows, create an ArgoCD Application manifest:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: opsdeck
  namespace: argocd
spec:
  project: default
  source:
    repoURL: 'https://github.com/pixelotes/opsdeck.git'
    targetRevision: HEAD
    path: helm/
    helm:
      valueFiles:
        - values.yaml
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: opsdeck
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

```
kubectl apply -f application.yaml
```


Copyright © 2024–2026 pixelotes — Elastic License

3.4.6 Scaling

OpsDeck can run multiple replicas behind the Kubernetes Service load balancer. Ensure:

- External PostgreSQL (internal PostgreSQL is not designed for multi-replica).
- Shared storage for attachments (ReadWriteMany PVC or S3-compatible storage).
- `SECRET_KEY` is identical across all replicas (from the shared Secret).

3.4.7 values.yaml reference

Key	Default	Description
`image.repository`	`pixelotes/opsdeck`	Container image
`image.tag`	`latest`	Image tag
`image.pullPolicy`	`IfNotPresent`	Pull policy
`service.type`	`ClusterIP`	Service type
`service.port`	`5000`	Service port
`database.type`	`internal`	`internal` or `external`
`persistence.logs.enabled`	`true`	Enable log persistence
`persistence.logs.size`	`500Mi`	Log volume size
`persistence.attachments.enabled`	`true`	Enable attachment persistence
`persistence.attachments.size`	`5Gi`	Attachment volume size

Copyright © 2024–2026 pixelotes — Elastic License

3.5 Manual Deployment

Deploy OpsDeck on a standard Linux server without containers, using Gunicorn behind Nginx.

3.5.1 System requirements

- Python 3.11+
- PostgreSQL 15+ (16 recommended)
- Nginx (for reverse proxy and TLS termination)
- System packages for WeasyPrint: `libpango-1.0-0`, `libpangocairo-1.0-0`, `libgdk-pixbuf2.0-0`, `libffi-dev`, `libcairo2`

3.5.2 Installation

1. System packages

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git nginx \
  libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf2.0-0 libffi-dev libcairo2 \
  postgresql postgresql-contrib
```

2. PostgreSQL setup

```
sudo -u postgres psql -c "CREATE USER opsdeck WITH PASSWORD 'your-db-password';"
sudo -u postgres psql -c "CREATE DATABASE opsdeck OWNER opsdeck;"
```

3. Application setup

```
cd /opt
sudo git clone https://github.com/pixelotes/opsdeck.git
cd opsdeck

python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Configuration

```
cp .env.example .env
```

Edit `.env`:

```
SECRET_KEY=<64-char random string>
DATABASE_URL=postgresql://opsdeck:your-db-password@localhost:5432/opsdeck
FLASK_DEBUG=0
SESSION_COOKIE_SECURE=True
DEFAULT_ADMIN_EMAIL=admin@yourcompany.com
DEFAULT_ADMIN_INITIAL_PASSWORD=<strong password>
```

5. Database initialization

```
flask db upgrade
flask init-db
flask seed-db-prod
```

6. Gunicorn

Test that the application starts:

```
gunicorn --bind 127.0.0.1:5000 --workers 4 --timeout 120 run:app
```


Copyright © 2024–2026 pixelotes — Elastic License

3.5.3 Systemd service

Create `/etc/systemd/system/opsdeck.service`:

```
[Unit]
Description=OpsDeck Application
After=network.target postgresql.service

[Service]
User=www-data
Group=www-data
WorkingDirectory=/opt/opsdeck
Environment="PATH=/opt/opsdeck/venv/bin"
EnvironmentFile=/opt/opsdeck/.env
ExecStart=/opt/opsdeck/venv/bin/gunicorn --bind 127.0.0.1:5000 --workers 4 --timeout 120 run:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable opsdeck
sudo systemctl start opsdeck
```

3.5.4 Nginx reverse proxy

Create `/etc/nginx/sites-available/opsdeck`:

```
server {
    listen 443 ssl;
    server_name opsdeck.yourcompany.com;

    ssl_certificate /etc/letsencrypt/live/opsdeck.yourcompany.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/opsdeck.yourcompany.com/privkey.pem;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        alias /opt/opsdeck/src/static/;
        expires 30d;
    }
}

server {
    listen 80;
    server_name opsdeck.yourcompany.com;
    return 301 https://$host$request_uri;
}
```

```
sudo ln -s /etc/nginx/sites-available/opsdeck /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

3.5.5 File permissions

```
sudo chown -R www-data:www-data /opt/opsdeck/data
sudo chmod 750 /opt/opsdeck/data/attachments
```


Copyright © 2024–2026 pixelotes — Elastic License

3.6 Environment Variables

OpsDeck is configured entirely via environment variables. Set them in your OS, in a `.env` file in the project root (auto-loaded via python-dotenv), or in your container orchestrator.

3.6.1 General

Variable	Description	Default	Required
`SECRET_KEY`	Cryptographic signing key for sessions, CSRF, and credential encryption. Must be long and random.	`your-secret-key-change-this`	**Yes**
`FLASK_APP`	Flask application entry point.	`run:app`	Yes
`FLASK_DEBUG`	Enable debug mode. `1` for development, `0` for production.	`0`	No

3.6.2 Database

Variable	Description	Default	Required
`DATABASE_URL`	PostgreSQL connection URI. Format: `postgresql://user:pass@host:port/dbname`	—	**Yes**

3.6.3 Admin initialization

These are used only on the first run to create the initial admin user.

Variable	Description	Default
`DEFAULT_ADMIN_EMAIL`	Admin user email.	`admin@example.com`
`DEFAULT_ADMIN_INITIAL_PASSWORD`	Admin user initial password. Changed on first login.	`admin123`

3.6.4 Email / SMTP

Variable	Description	Default
`SMTP_SERVER`	SMTP server hostname.	`smtp.gmail.com`
`SMTP_PORT`	SMTP server port.	`587`
`EMAIL_USERNAME`	SMTP authentication username (usually an email).	`''`
`EMAIL_PASSWORD`	SMTP authentication password or app password.	`''`

3.6.5 Notifications

Variable	Description	Default
`WEBHOOK_URL`	External webhook URL for notifications (Slack, Discord).	`''`

Copyright © 2024–2026 pixelotes — Elastic License

3.6.6 Authentication

Variable	Description	Default
`GOOGLE_OAUTH_CLIENT_ID`	Google OAuth client ID for SSO.	`''`
`GOOGLE_OAUTH_CLIENT_SECRET`	Google OAuth client secret.	`''`
`OAUTHLIB_INSECURE_TRANSPORT`	Allow OAuth over HTTP. **Development only.**	—
`MFA_ENABLED`	Enable TOTP multi-factor authentication.	`False`

3.6.7 Session and cookies

Variable	Default	Production recommendation
`SESSION_COOKIE_SECURE`	`False`	`True`
`SESSION_COOKIE_HTTPONLY`	`True`	`True`
`SESSION_COOKIE_SAMESITE`	`Lax`	`Lax` or `Strict`

3.6.8 Enterprise features

Variable	Description	Default
`ENTERPRISE_ENABLED`	Enable enterprise-tier features.	`False`
`SEED_DEMO_DATA`	Seed demo data on first run. Development only.	`False`

Minimal production `.env`

```
SECRET_KEY=<64-char random string>
DATABASE_URL=postgresql://opsdeck:password@db:5432/opsdeck
FLASK_DEBUG=0
SESSION_COOKIE_SECURE=True
DEFAULT_ADMIN_EMAIL=admin@yourcompany.com
DEFAULT_ADMIN_INITIAL_PASSWORD=<strong password>
```


Copyright © 2024–2026 pixelotes — Elastic License

3.7 Database Management

OpsDeck uses Flask-Migrate (Alembic) for database schema versioning with a sequential numbering convention.

3.7.1 Migration conventions

Migrations are stored in `migrations/versions/` with sequential numbering:

```
migrations/versions/
├── 001_initial.py
├── 002_add_notes_field.py
├── 003_add_risk_categories.py
└── ...
```

This provides human-readable ordering and enforces a linear migration history. No branching migrations.

3.7.2 Running migrations

```
# Apply all pending migrations
flask db upgrade

# Check current migration version
flask db current

# View migration history
flask db history
```

For Docker deployments, migrations run automatically on container startup via `entrypoint.sh`. For manual or Kubernetes deployments, run migrations explicitly before starting the application.

3.7.3 Creating new migrations

After modifying SQLAlchemy models:

```
# Auto-generate a migration
flask db migrate -m "description of changes"

# Rename the generated file to follow sequential numbering
# e.g., rename random hex to 015_add_certificate_tracking.py

# Review the generated migration
# Verify upgrade() and downgrade() functions

# Apply
flask db upgrade
```

> **Warning**
>
> Always review auto-generated migrations. Alembic may miss some changes (table renames, data migrations) or generate incorrect operations.

3.7.4 Rollback

```
# Roll back one migration
flask db downgrade -1

# Roll back to a specific revision
flask db downgrade 010
```


Copyright © 2024–2026 pixelotes — Elastic License

3.7.5 Database initialization

On a fresh database:

```
flask db upgrade     # Apply all migrations
flask init-db        # Create base tables and configuration
flask seed-db-prod   # Create admin user and seed reference data
```


Copyright © 2024–2026 pixelotes — Elastic License

3.8 Backup & Restore

OpsDeck stores data in two places: PostgreSQL (all application data) and the filesystem (attachments and uploaded files). Both must be backed up together to ensure a consistent restore.

3.8.1 What to back up

Component	Location	Method
Database	PostgreSQL	`pg_dump`
Attachments	`/app/data/attachments` (container) or `./data/attachments` (host)	File copy / tar
Configuration	`.env` file	File copy

3.8.2 Automated backup script

```
#!/bin/bash
# backup-opsdeck.sh
set -euo pipefail

BACKUP_DIR="/backups/opsdeck"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
RETENTION_DAYS=30

mkdir -p "$BACKUP_DIR"

# Database backup
echo "Backing up database..."
docker-compose exec -T db pg_dump -U opsdeck opsdeck \
  | gzip > "$BACKUP_DIR/db_${TIMESTAMP}.sql.gz"

# Attachments backup
echo "Backing up attachments..."
tar -czf "$BACKUP_DIR/attachments_${TIMESTAMP}.tar.gz" ./data/attachments

# Cleanup old backups
find "$BACKUP_DIR" -name "*.gz" -mtime +${RETENTION_DAYS} -delete

echo "Backup complete: $BACKUP_DIR/*_${TIMESTAMP}.*"
```

Schedule via cron:

```
0 2 * * * /opt/scripts/backup-opsdeck.sh >> /var/log/opsdeck-backup.log 2>&1
```

3.8.3 Restore procedure

1. Stop the application

```
docker-compose stop web
```

2. Restore the database

```
# Drop and recreate the database
docker-compose exec db psql -U opsdeck -c "DROP DATABASE opsdeck;"
docker-compose exec db psql -U opsdeck -c "CREATE DATABASE opsdeck;"

# Restore from backup
gunzip -c /backups/opsdeck/db_20260324_020000.sql.gz \
  | docker-compose exec -T db psql -U opsdeck opsdeck
```

3. Restore attachments

```
rm -rf ./data/attachments
tar -xzf /backups/opsdeck/attachments_20260324_020000.tar.gz
```


Copyright © 2024–2026 pixelotes — Elastic License

4. Restart

```
docker-compose start web
```

3.8.4 Kubernetes backup

For Helm deployments, adapt the strategy:

- Use `kubectl exec` instead of `docker-compose exec` to run `pg_dump`.
- For external PostgreSQL (RDS/Aurora), use the provider's native backup (automated snapshots, point-in-time recovery).
- For attachment PVCs, use your storage provider's snapshot mechanism or Velero for volume-level backups.

3.8.5 Verification

After any restore, verify:

1. Application starts without migration errors.
2. Dashboard loads with correct data.
3. Attachments are accessible (open any asset or policy with uploaded files).
4. Audit log contains expected history.

Copyright © 2024–2026 pixelotes — Elastic License

3.9 Upgrading

Procedure for upgrading OpsDeck to a new version.

3.9.1 Version compatibility

OpsDeck follows semantic versioning. Database migrations handle schema changes between versions. Always read the Changelog before upgrading to understand breaking changes.

3.9.2 Upgrade procedure

1. Backup

Always back up before upgrading:

```
# Database
docker-compose exec -T db pg_dump -U opsdeck opsdeck | gzip > backup_pre_upgrade.sql.gz

# Attachments
tar -czf attachments_pre_upgrade.tar.gz ./data/attachments
```

2. Pull new version

Docker Manual

```
docker-compose pull
# or rebuild if using local Dockerfile:
docker-compose build --no-cache
```

```
cd /opt/opsdeck
git fetch origin
git checkout v0.x.y  # Target version tag
source venv/bin/activate
pip install -r requirements.txt
```

3. Run migrations

Docker Manual

Migrations run automatically on container startup via `entrypoint.sh`.

```
flask db upgrade
```

4. Restart

Docker Manual

```
docker-compose up -d
```

```
sudo systemctl restart opsdeck
```

5. Verify

- Check application logs for migration errors.
- Verify the dashboard loads correctly.
- Confirm the version number in the UI footer.

Copyright © 2024–2026 pixelotes — Elastic License

3.9.3 Dependency updates

Python and JavaScript dependencies are updated periodically:

```
# Python — use the provided script
bash update-deps.sh

# JavaScript
bash update-node-deps.sh
```

Both scripts update dependencies and run tests to verify compatibility.

3.9.4 Rollback plan

If the upgrade fails:

1. Stop the application.
2. Restore the database from backup.
3. Restore attachments from backup.
4. Roll back to the previous version (Docker image tag or git checkout).
5. Restart.

Copyright © 2024–2026 pixelotes — Elastic License

3.10 Monitoring & Logging

OpsDeck uses ECS-format structured logging and provides a health check endpoint for monitoring integration.

3.10.1 Structured logging

All application logs are emitted in ECS (Elastic Common Schema) format using the `ecs-logging` library. This makes them directly ingestible by Elasticsearch, Logstash, and Kibana.

Log entries include:

- **Timestamp** — ISO 8601 UTC.
- **Log level** — DEBUG, INFO, WARNING, ERROR.
- **Event metadata** — `event.action`, `event.category`, `event.outcome`.
- **User context** — `user.id`, `user.email` (when available).
- **Request context** — `source.ip`, `http.request.method`, `url.path`.

Log configuration

Log level is controlled via the `LOG_LEVEL` environment variable (default: `INFO`). Set to `DEBUG` for troubleshooting.

Logs are written to stdout by default (suitable for Docker/Kubernetes log collection). For file output, configure in the application or redirect via your process manager.

3.10.2 Health check endpoint

```
GET /health
```

Returns HTTP 200 with a JSON body if the application is healthy:

```
{
  "status": "healthy",
  "database": "connected",
  "scheduler": "running"
}
```

Checks performed:

- Database connectivity (simple query).
- APScheduler status (running vs. stopped).

Use this endpoint for:

- Docker health checks (`HEALTHCHECK` in Dockerfile or `docker-compose.yml`).
- Kubernetes liveness and readiness probes.
- External monitoring (Uptime Robot, Pingdom, etc.).

3.10.3 Integration with ELK

Since logs are already in ECS format, integration with Elasticsearch is straightforward:

1. Configure your log driver to ship logs to Logstash or Filebeat.
2. Logs parse natively without custom grok patterns.
3. Use Kibana to create dashboards for: API access patterns, authentication failures, error rates, and audit events.

Copyright © 2024–2026 pixelotes — Elastic License

3.10.4 APScheduler monitoring

Scheduled job execution is logged with:

- Job start and completion timestamps.
- Duration and outcome (success/failure).
- Error details with stack traces on failure.

Monitor for:

- Jobs that stop executing (scheduler died).
- Jobs with increasing duration (performance degradation).
- Repeated failures on the same job.

Copyright © 2024–2026 pixelotes — Elastic License

3.11 Security Hardening

Production security checklist and configuration guidance.

3.11.1 Pre-deployment checklist

- [ ] `SECRET_KEY` is a unique, random string (at least 64 characters).
- [ ] `FLASK_DEBUG=0` is set.
- [ ] `SESSION_COOKIE_SECURE=True` (requires HTTPS).
- [ ] `SESSION_COOKIE_HTTPONLY=True` (default).
- [ ] `SESSION_COOKIE_SAMESITE=Lax` or `Strict`.
- [ ] Default admin password has been changed.
- [ ] Database password is strong and unique.
- [ ] TLS is configured on the reverse proxy.
- [ ] `OAUTHLIB_INSECURE_TRANSPORT` is **not** set.

3.11.2 Content Security Policy

flask-talisman enforces a CSP that restricts script and style sources. The default policy allows:

- Scripts and styles from `'self'` and the specific CDN sources used by vendor libraries.
- Inline styles for Bootstrap components (via nonce or `'unsafe-inline'` where unavoidable).
- No inline scripts.

Review and tighten the CSP for your environment. Test with browser developer tools to catch blocked resources.

3.11.3 Rate limiting

Flask-Limiter protects against:

- **Login brute-force** — limited attempts per IP per time window.
- **API abuse** — per-token rate limits on API endpoints.
- **Form spam** — general rate limits on POST endpoints.

Configure limits via environment variables or in the application configuration. Defaults are conservative — adjust based on expected legitimate traffic.

3.11.4 OAuth / SSO setup

For Google OAuth:

1. Create OAuth credentials in Google Cloud Console.
2. Set authorized redirect URI to `https://your-domain/login/google/authorized`.
3. Configure `GOOGLE_OAUTH_CLIENT_ID` and `GOOGLE_OAUTH_CLIENT_SECRET`.
4. Ensure `OAUTHLIB_INSECURE_TRANSPORT` is **not** set in production.

Users must already exist in OpsDeck — OAuth authenticates but does not auto-create accounts.

Copyright © 2024–2026 pixelotes — Elastic License

3.11.5 Secret rotation

Secret	Rotation impact	Procedure
`SECRET_KEY`	Invalidates all sessions and encrypted credentials	Update key, restart app, re-encrypt credentials
Database password	Requires config update	Update in PostgreSQL and `.env`, restart app
API tokens	Per-user, no global impact	Revoke from user profile, generate new token
OAuth secrets	Breaks SSO until updated	Update in Google Console and `.env`

Danger

Changing `SECRET_KEY` renders all Fernet-encrypted credentials unreadable. Export credential vault contents **before** rotating the key.

3.11.6 Network security

- Place the application behind a reverse proxy (Nginx, Traefik) — never expose Gunicorn directly.
- Isolate PostgreSQL from public network access.
- Use firewall rules to restrict access to necessary ports only.
- For Kubernetes, use NetworkPolicies to limit pod-to-pod communication.

3.11.7 Known IP tracking

OpsDeck records the IP addresses used by each user (`UserKnownIP`). This enables:

- Detection of logins from new/unusual locations.
- Audit trail of access origins.
- Future: alerting on logins from unknown IPs.

Copyright © 2024–2026 pixelotes — Elastic License

4. Architecture

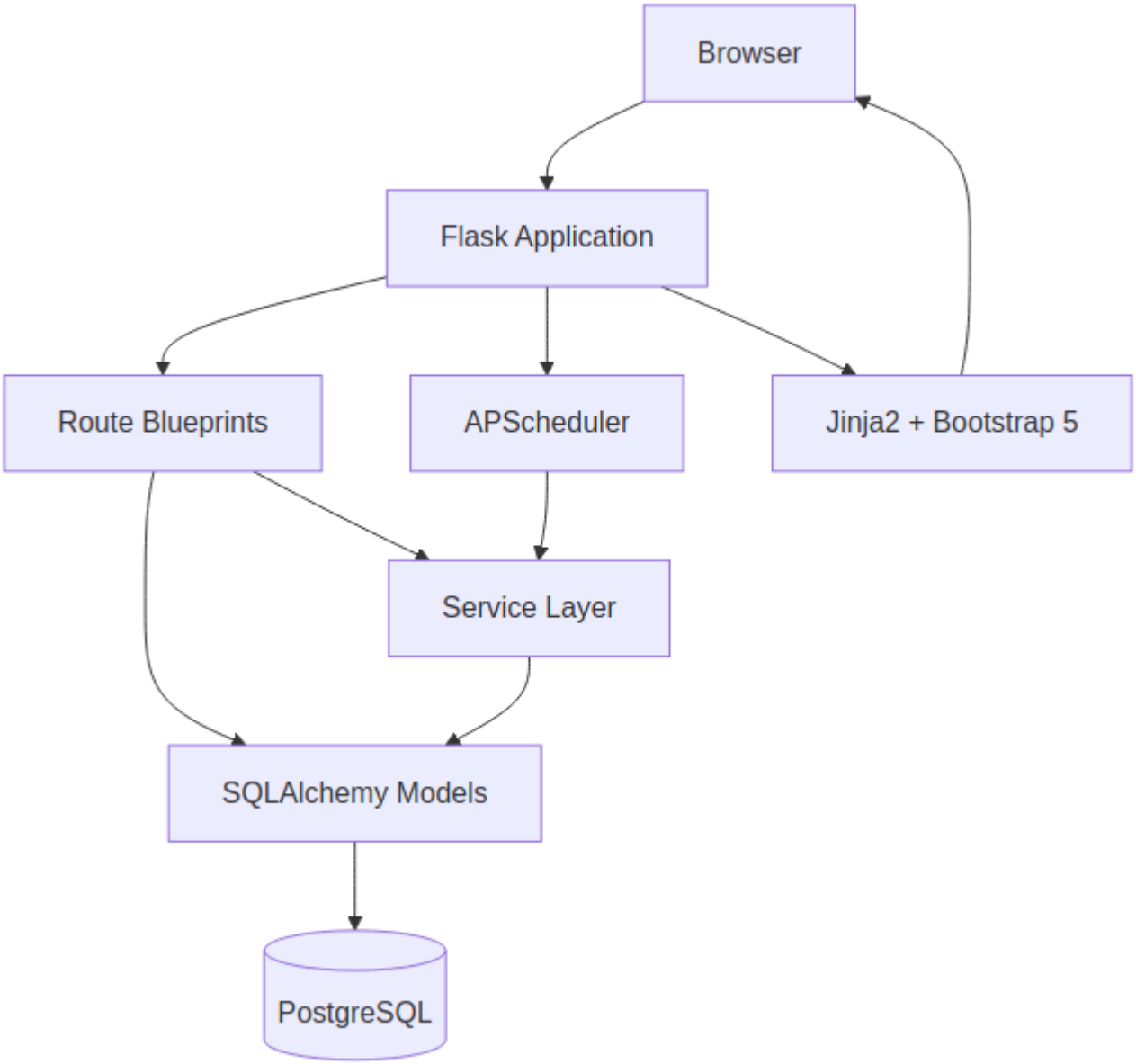
4.1 Architecture Reference

This section documents the technical architecture of OpsDeck, including the system design, data model, library choices, and the reasoning behind key decisions.

4.1.1 Overview

OpsDeck is a Python/Flask monolith backed by PostgreSQL. It follows a layered architecture with clear separation between models, routes, services, and utilities. The frontend uses server-rendered Jinja2 templates with Bootstrap 5, enhanced by client-side JavaScript libraries for rich interactions.

Copyright © 2024–2026 pixelotes — Elastic License



4.1.2 Sections

Document	Covers
System Overview	High-level architecture, request flow, layer responsibilities
Data Model	Entity relationships, core tables, polymorphic patterns
Permissions & RBAC	Role-based access control, module-level permissions, caching
Decision Records	ADRs — why Flask, why monolith, why Elastic License, etc.
Dependencies	Every Python and JS dependency with justification
Frontend Stack	Bootstrap 5, Chart.js, Tom Select, Mermaid, DataTables
API Design	REST conventions, authentication, marshmallow schemas
Scheduler & Jobs	APScheduler setup, recurring jobs, compliance drift
Security Model	Auth flow, session management, encryption, CSP, rate limiting

Copyright © 2024–2026 pixelotes — Elastic License

4.2 System Overview

OpsDeck is a Python/Flask monolith backed by PostgreSQL, designed as a single deployable unit that covers IT operations, governance, compliance, and vendor management.

4.2.1 Design philosophy

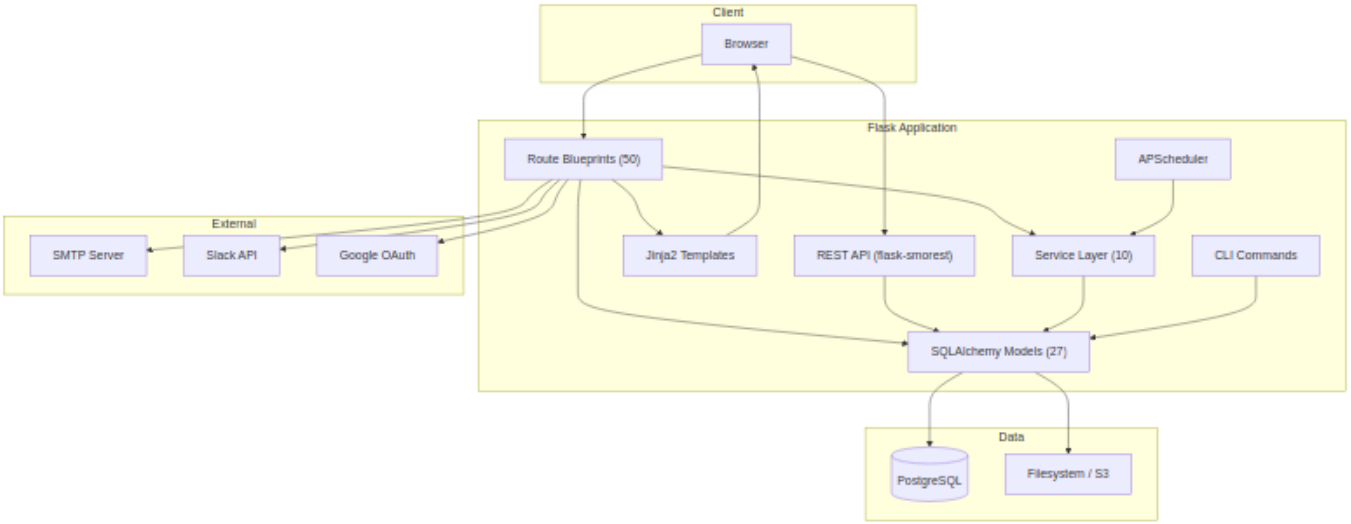
Three principles drive every architectural choice:

Compliance-first design. All features are built with audit requirements in mind. The system maintains comprehensive logs, enforces data integrity, and provides clear audit trails for all operations. Immutable audit snapshots, soft deletes, and centralized logging are defaults, not afterthoughts.

Integrated data model. Rather than treating assets, users, services, and compliance as separate domains, OpsDeck uses an interconnected data model where entities can be linked across domains via polymorphic `ComplianceLink` and `Link` records. This enables traceability and impact analysis — you can trace from a business service down to the hardware it runs on, the vendor who supplies it, and the compliance controls that govern it.

Extensibility without complexity. The platform uses a modular blueprint architecture. Each functional area has its own route file, model file, and template directory. Features can be added or disabled without affecting core functionality. Configuration is environment-based rather than database-driven where possible.

4.2.2 High-level architecture



4.2.3 Request lifecycle

A typical request flows through these layers:

1. **Gunicorn** accepts the HTTP connection and forwards to the Flask WSGI app.
2. **Flask middleware** runs: session loading (Flask-Login), CSRF validation (Flask-WTF), security headers (flask-talisman), rate limiting (Flask-Limiter).
3. **Route blueprint** matches the URL. The route function checks permissions via the `@permission_required` decorator, which queries the permissions cache.
4. **Business logic** runs — either inline in the route or delegated to a service (e.g., `compliance_drift_service`, `uar_service`, `finance_service`).
5. **SQLAlchemy models** interact with PostgreSQL. The audit listener (`utils/audit_listener.py`) captures all INSERT/UPDATE/DELETE events for the audit log.
6. **Jinja2 template** renders the response using Bootstrap 5 layout inheritance. For API routes, marshmallow schemas serialize the response to JSON.

Copyright © 2024–2026 pixelotes — Elastic License

4.2.4 Module structure

```
src/
├── __init__.py             # App factory, blueprint registration, config
├── api.py                  # REST API setup (flask-smorest)
├── cli.py                  # CLI commands: import, seed, admin tasks
├── extensions.py           # Flask extension instances
├── schemas.py              # Marshmallow schemas for API serialization
├── notifications.py        # Email and webhook notification engine
├── seeder.py               # Demo data seeder
├── seeder_prod.py          # Production bootstrap seeder
│
├── models/                 # SQLAlchemy models (27 files, ~5,600 lines)
│   ├── core.py             # Shared: Attachment, Tag, Link, CostCenter, CustomFields
│   ├── auth.py             # User, Group, OrgChartSnapshot, KnownIP
│   ├── assets.py           # Asset, Peripheral, Software, Location, Maintenance, Disposal
│   ├── security.py         # Framework, Control, Risk, Incident, ComplianceLink/Rule
│   ├── audits.py           # ComplianceAudit, AuditControlItem, AuditControlLink
│   ├── uar.py              # UARComparison, UARExecution, UARFinding
│   ├── procurement.py      # Supplier, Subscription, Contract, Purchase, Budget
│   ├── services.py         # BusinessService, ServiceComponent
│   ├── policy.py           # Policy, PolicyVersion, PolicyAcknowledgement
│   ├── training.py         # Course, CourseAssignment, CourseCompletion
│   ├── onboarding.py       # OnboardingPack, ProcessTemplate, ProcessItem
│   ├── credentials.py      # Credential, CredentialSecret (Fernet-encrypted)
│   ├── risk_assessment.py  # RiskAssessment, RiskAssessmentItem, Evidence
│   ├── ...                 # hiring, bcdr, crm, finance, certificates, etc.
│   └── permissions.py      # Module, Permission, AccessLevel enum
│
├── routes/                 # Flask blueprints (50 files, ~15,000 lines)
│   ├── main.py             # Dashboard, health check, universal search
│   ├── assets.py           # CRUD + lifecycle transitions
│   ├── compliance.py       # Compliance dashboard, drift, evidence linking
│   ├── audits.py           # Audit creation, cloning, locking, export
│   ├── ...                 # One file per functional module
│   └── search.py           # Universal faceted search
│
├── services/               # Business logic layer (10 files, ~3,000 lines)
│   ├── compliance_service.py       # Score calculation, status aggregation
│   ├── compliance_drift_service.py  # Snapshot capture, drift detection
│   ├── uar_service.py              # Comparison execution, finding generation
│   ├── finance_service.py          # Budget tracking, forecast, exchange rates
│   ├── permissions_service.py      # RBAC resolution
│   ├── permissions_cache.py        # In-memory permission caching
│   ├── search_service.py           # Cross-entity search
│   ├── audit_export_service.py     # Evidence package generation
│   └── slack_service.py            # Slack notification integration
│
├── utils/                  # Shared utilities (9 files)
│   ├── audit_listener.py   # SQLAlchemy event listener for audit logging
│   ├── uar_engine.py       # Core diff engine for UAR comparisons
│   ├── dependency_graph.py  # Service dependency resolution
│   ├── differ.py           # DeepDiff wrapper for change detection
│   ├── helpers.py          # Shared helper functions
│   ├── timezone_helper.py  # Timezone-aware datetime handling
│   └── logger.py           # ECS-format structured logging setup
│
├── templates/              # Jinja2 templates (~40,000 lines)
│   ├── layout.html         # Base layout with Bootstrap 5
│   ├── components/         # Reusable UI components (modals, forms, tables)
│   └── [module]/           # One directory per functional module
│
└── static/                 # CSS, JS, vendor assets
    ├── css/
    ├── js/
    └── vendor/             # Copied from node_modules via copy-assets.js
```


Copyright © 2024–2026 pixelotes — Elastic License

4.2.5 Key patterns

Polymorphic linking

The `ComplianceLink` model connects any "linkable" entity (asset, policy, service, supplier, etc.) to any framework control. This is how evidence is linked to compliance requirements without requiring a separate junction table per entity type.

Similarly, the `Link` model provides generic entity-to-entity linking for cross-domain relationships (e.g., linking a risk to an asset, or a service to a contract).

Custom properties

The `CustomPropertiesMixin` allows any model that includes it (currently `Asset`, `User`, `Peripheral`) to have arbitrary key-value custom fields defined at the organization level via `CustomFieldDefinition`.

Audit logging

The `audit_listener.py` hooks into SQLAlchemy's `after_insert`, `after_update`, and `after_delete` events. Every mutation to a tracked model generates an `AuditLog` record with the user, timestamp, operation type, and a JSON diff of changed fields (powered by DeepDiff).

Permission caching

The RBAC system resolves permissions per user per module. To avoid repeated database queries on every request, resolved permissions are cached in-memory via `permissions_cache.py` and invalidated when roles or group memberships change.

Copyright © 2024–2026 pixelotes — Elastic License

4.3 Data Model

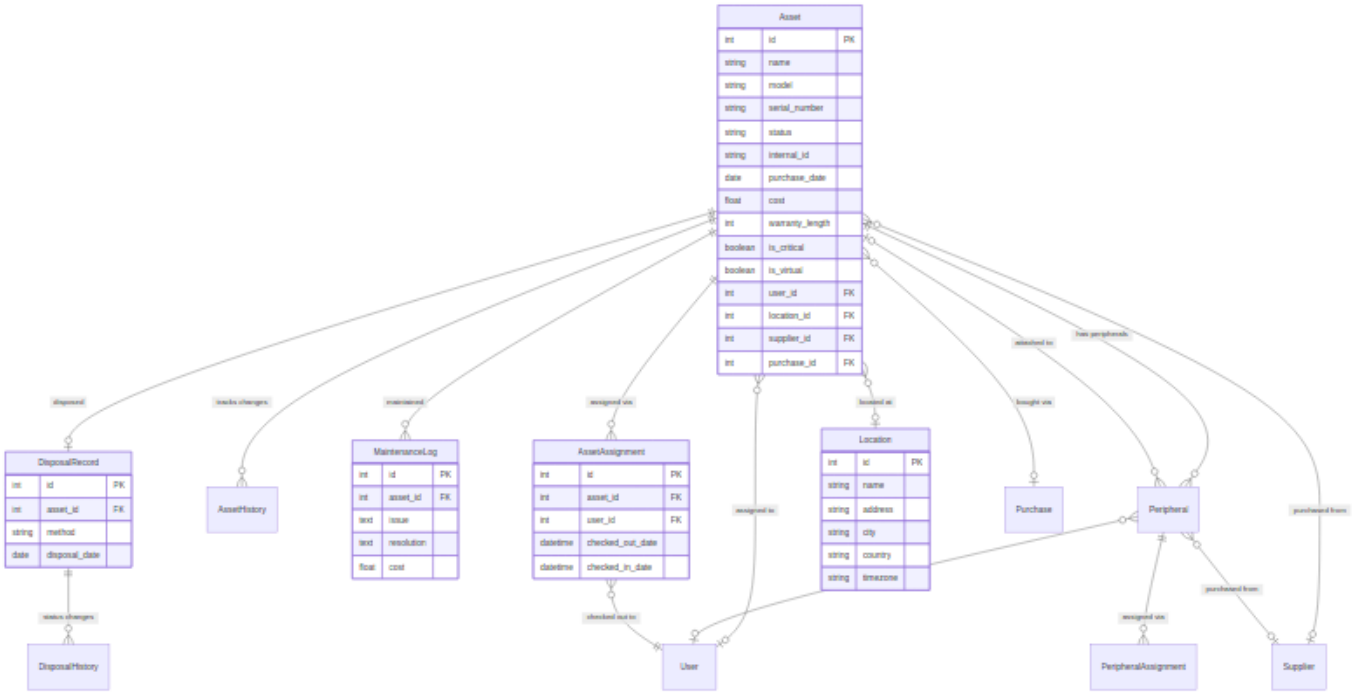
OpsDeck uses SQLAlchemy as its ORM with PostgreSQL as the sole supported database. The data model is organized into domain-specific model files with shared patterns for cross-cutting concerns.

4.3.1 Overview

The full schema has 80+ tables across 27 model files. Rather than one monolithic diagram, the relationships are documented below by domain.

4.3.2 Assets & hardware

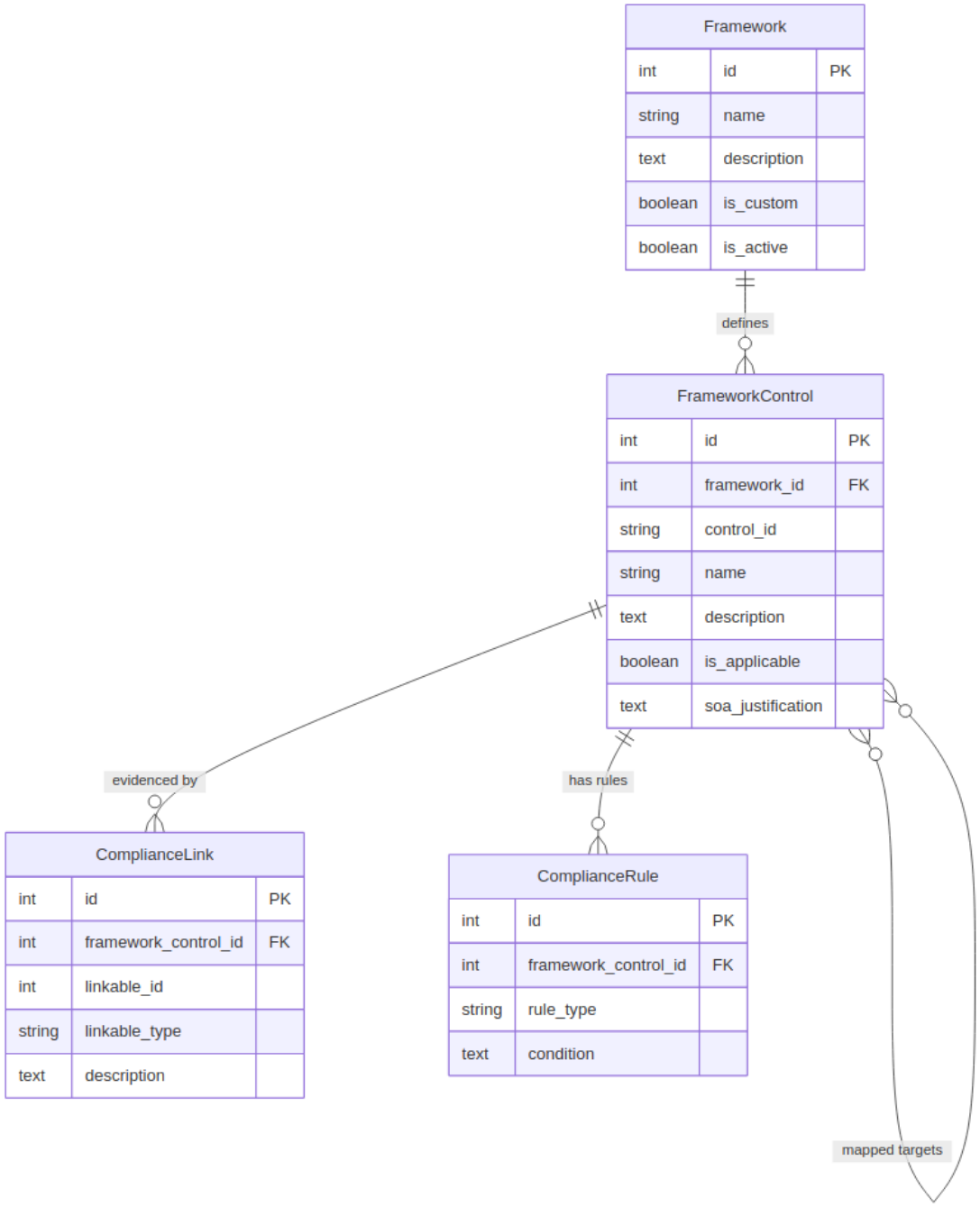
Tracks hardware lifecycle from procurement to disposal, with assignment history, maintenance, and peripherals.



4.3.3 Compliance & frameworks

The core governance engine: frameworks define requirements, controls define individual rules, and compliance links connect evidence from any entity to any control.

Copyright © 2024–2026 pixelotes — Elastic License



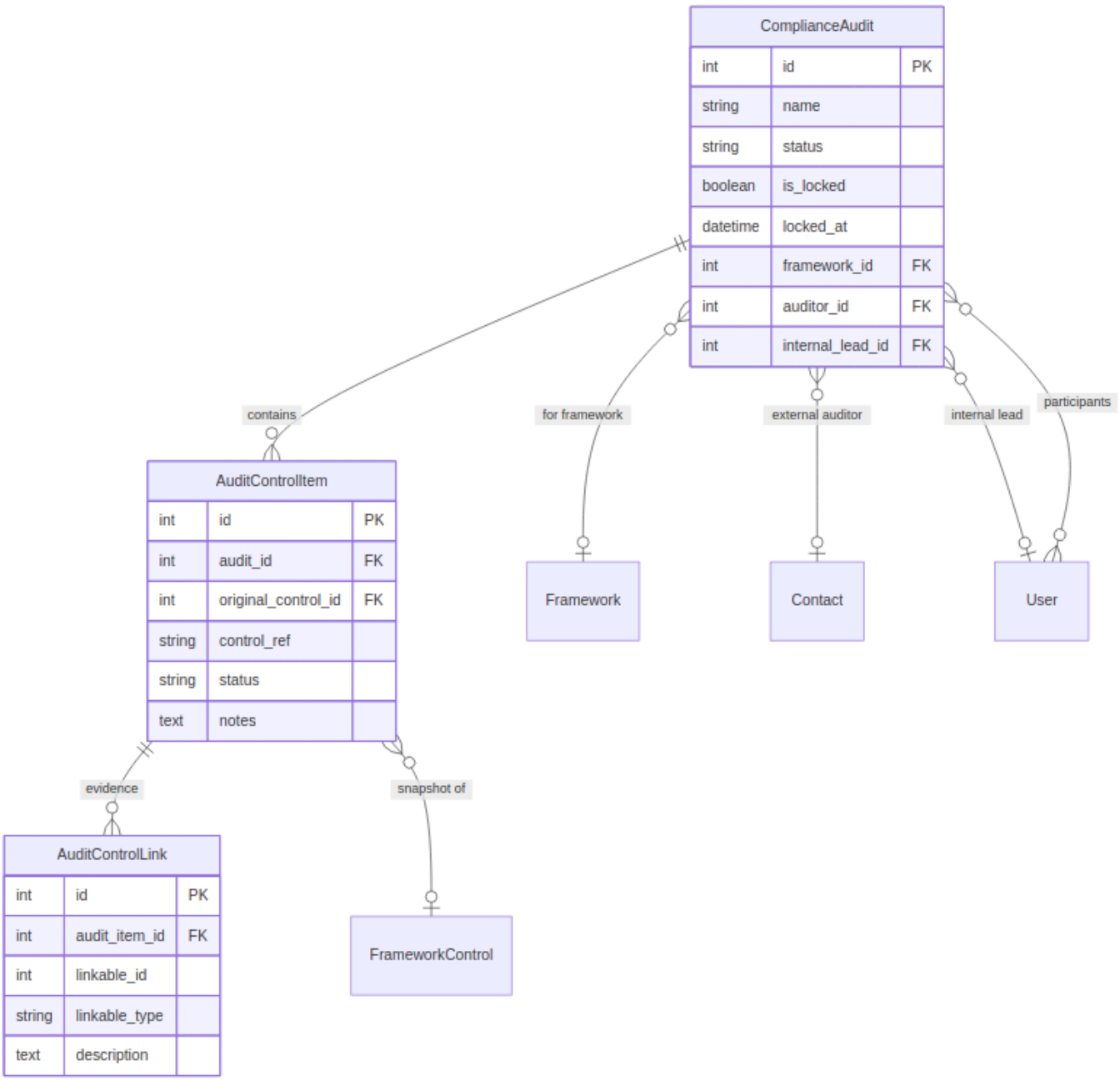
Polymorphic linking

`ComplianceLink.linkable_type` stores the entity class name (Asset, Policy, Supplier, etc.) and `linkable_id` stores its primary key. This avoids a separate junction table per entity type.

4.3.4 Audits

Point-in-time compliance snapshots with immutable evidence for auditors.

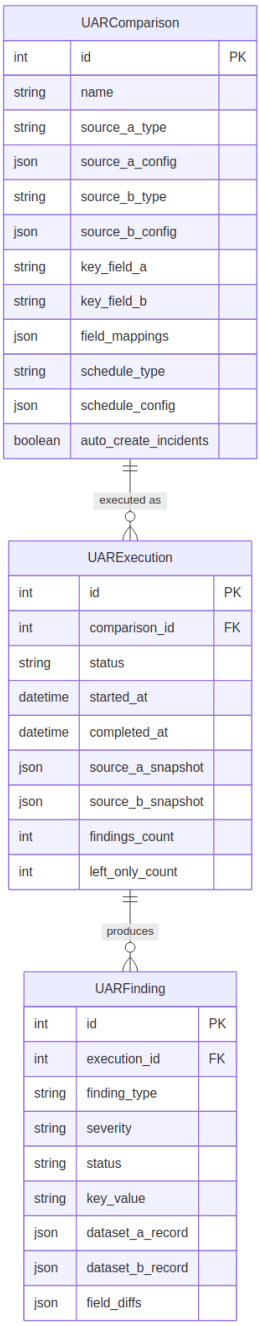
Copyright © 2024–2026 pixelotes — Elastic License



4.3.5 User access reviews (UAR)

Automated comparison engine for detecting access discrepancies between systems.

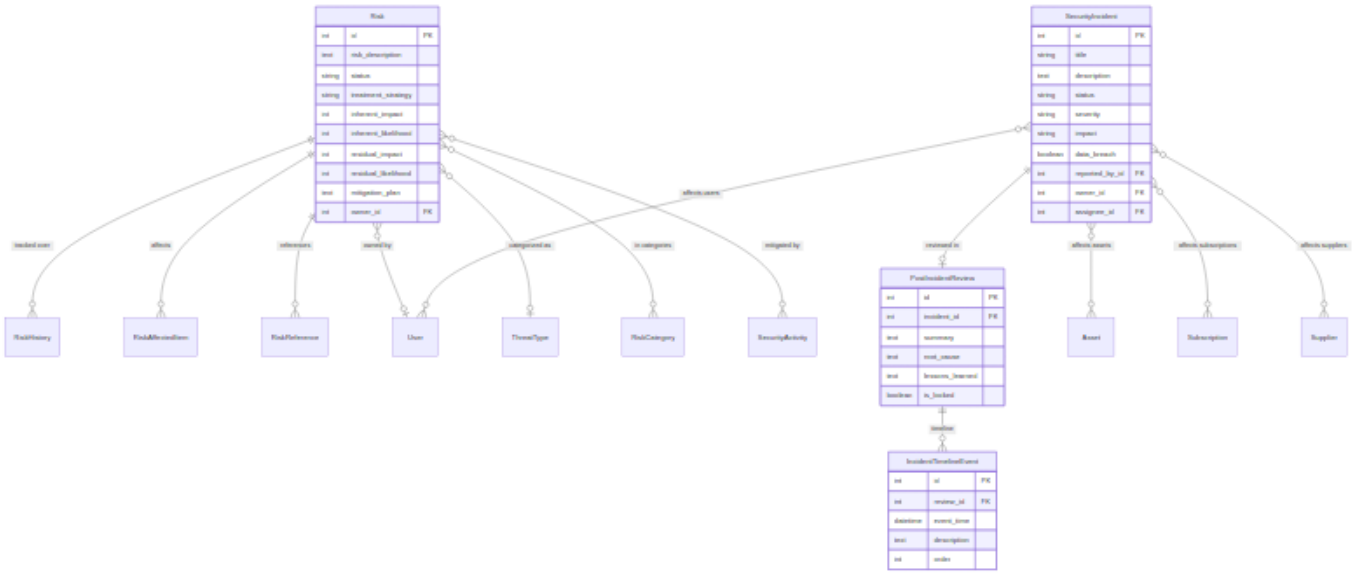
Copyright © 2024–2026 pixelotes — Elastic License



4.3.6 Risk & security

Risk register, security incidents with timeline tracking, and post-incident reviews.

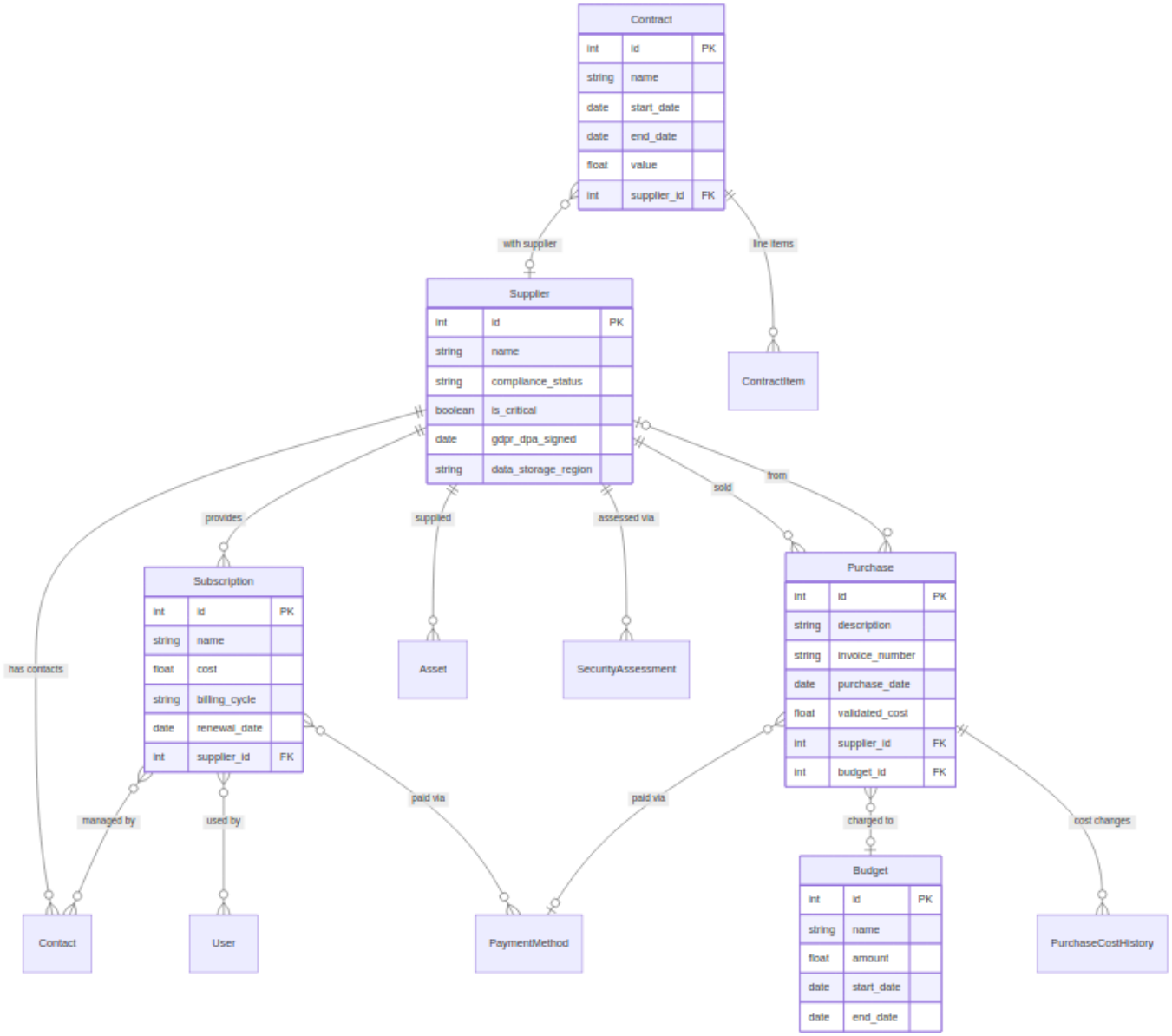
Copyright © 2024–2026 pixelotes — Elastic License



4.3.7 Procurement & vendors

Supplier management, contracts, subscriptions, purchases, and budgets.

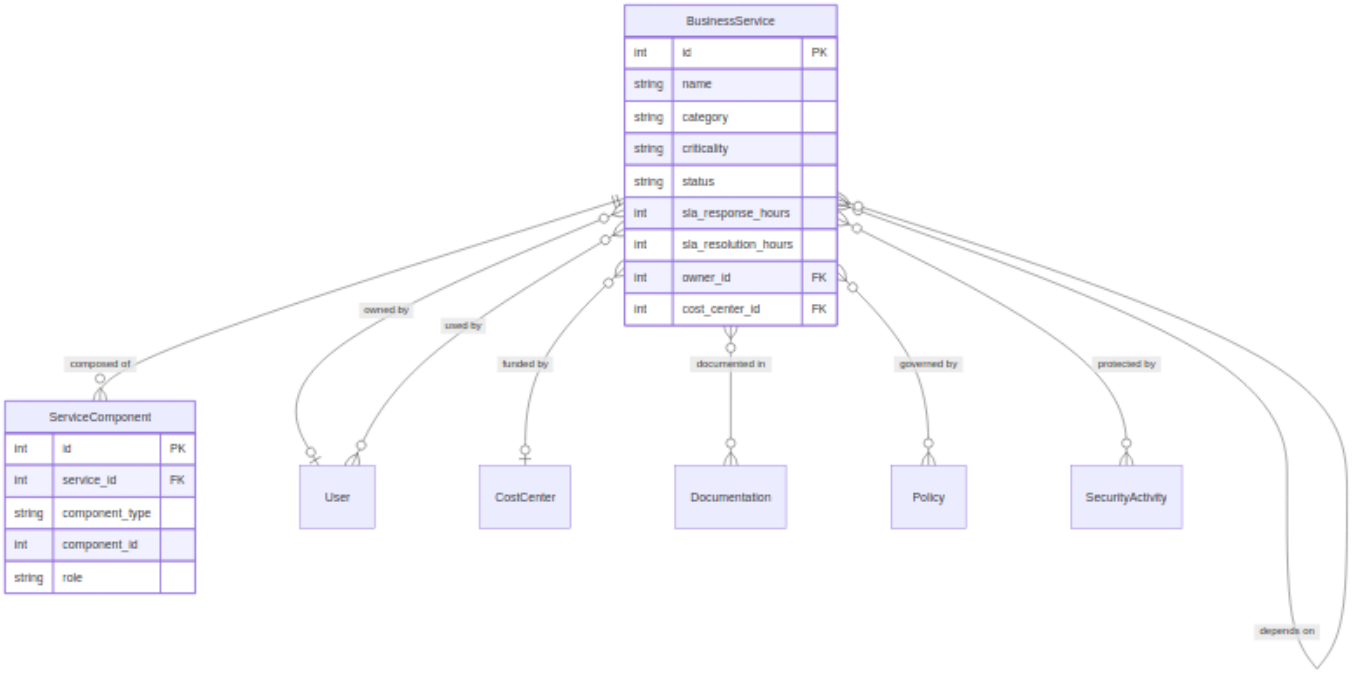
Copyright © 2024–2026 pixelotes — Elastic License



4.3.8 Service catalog

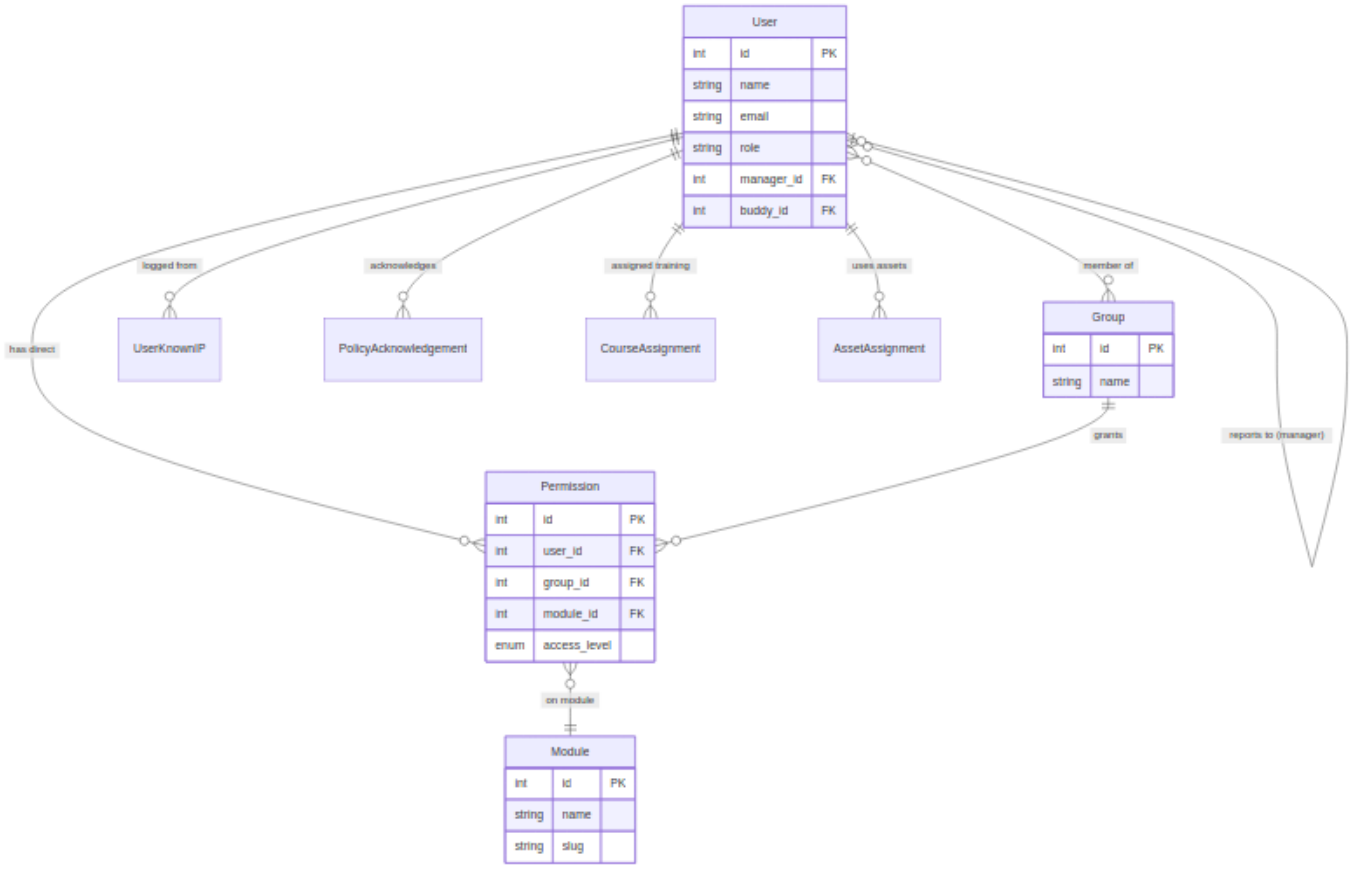
Business services mapped to technical components with dependency tracking.

Copyright © 2024–2026 pixelotes — Elastic License



4.3.9 People & authentication

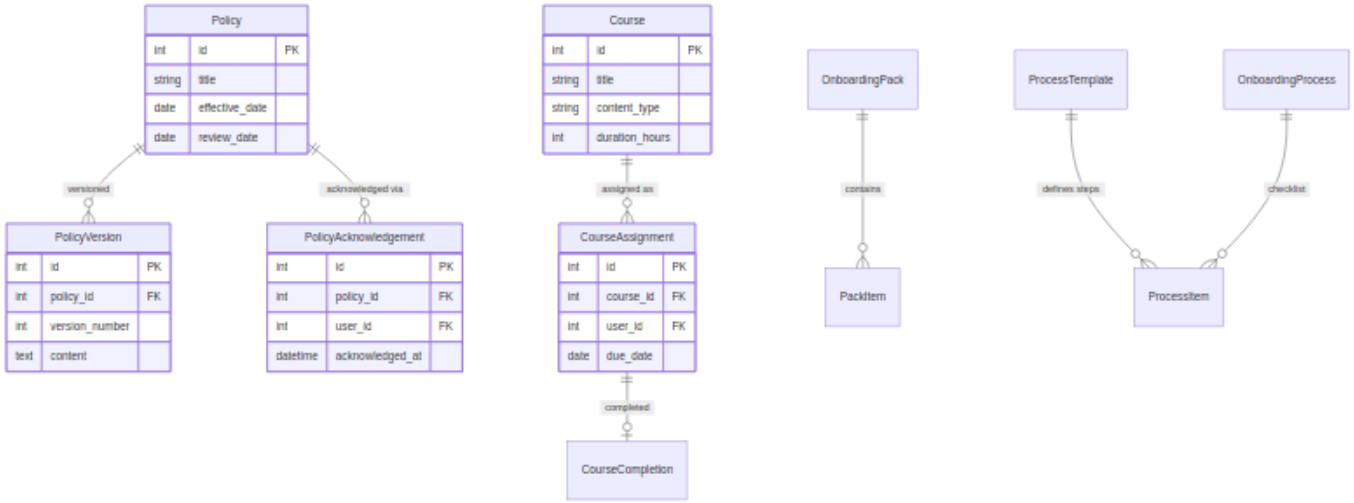
Users, groups, RBAC permissions, org chart, and known IP tracking.



Copyright © 2024–2026 pixelotes — Elastic License

4.3.10 Policies, training & onboarding

Governance workflows: policy acknowledgment, training tracking, and employee lifecycle.



4.3.11 Cross-cutting patterns

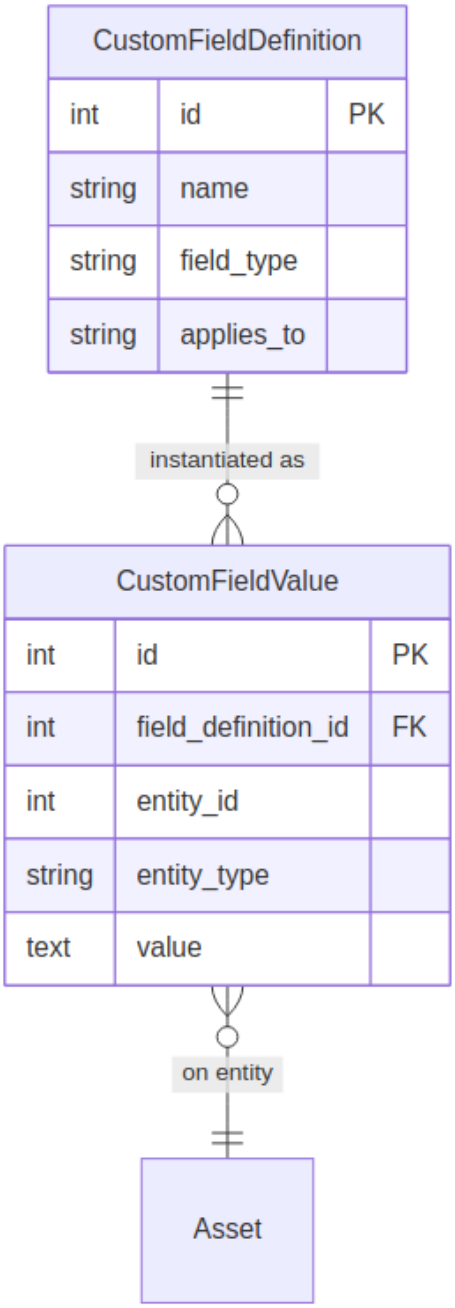
Polymorphic linking

Both `ComplianceLink` and `Link` use a `linkable_type` + `linkable_id` pattern to connect any entity to compliance controls or to other entities without per-type junction tables. `RiskAffectedItem` and `RiskReference` follow the same pattern.

CustomPropertiesMixin

Models that include this mixin (`Asset`, `User`, `Peripheral`) support arbitrary custom fields:

Copyright © 2024–2026 pixelotes — Elastic License



Audit logging

Every mutation generates an `AuditLog` record via the SQLAlchemy event listener:

Copyright © 2024–2026 pixelotes — Elastic License

Column	Content
`user_id`	Who made the change
`action`	INSERT / UPDATE / DELETE
`table_name`	Affected table
`record_id`	Affected record
`timestamp`	UTC timestamp
`changes`	JSON diff (DeepDiff output)

Soft deletes and history

Most entities use `is_archived` flags rather than physical deletion. History tables (`AssetHistory`, `RiskHistory`, `CostHistory`, `DisposalHistory`) capture point-in-time snapshots for trending and timeline visualization.

Copyright © 2024–2026 pixelotes — Elastic License

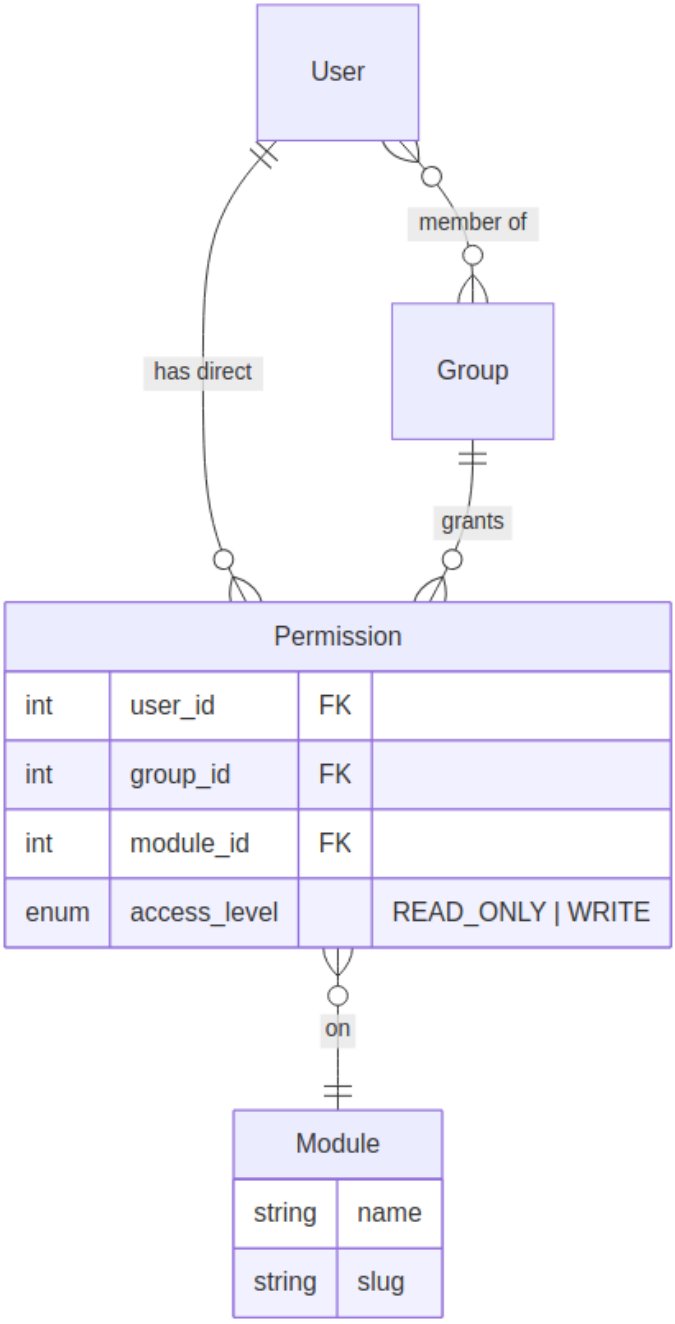
4.4 Permissions & RBAC

OpsDeck implements a module-level role-based access control system with group inheritance and in-memory caching.

4.4.1 Permission model

Permissions are defined per module (assets, compliance, risk, etc.) with two access levels:

Copyright © 2024–2026 pixelotes — Elastic License



The `AccessLevel` enum has two values:

Level	Grants
`READ_ONLY`	View access to module data
`WRITE`	Create, update, and delete access (implies read)

Copyright © 2024–2026 pixelotes — Elastic License

4.4.2 Resolution algorithm

When a request hits a `@permission_required(module, level)` decorator:

1. Check the permission cache for the user.
2. If not cached, resolve permissions:
    - Load direct permissions (`Permission` records where `user_id` matches).
    - Load group permissions (from all groups the user belongs to).
    - Merge: for each module, take the highest access level (WRITE > READ_ONLY).
3. Cache the resolved permission map.
4. Check if the user has the required level on the requested module.

4.4.3 Group-based permissions

Groups simplify permission management for teams:

- Create a group (e.g., "IT Operations," "Compliance Team").
- Assign module permissions to the group.
- Add users as members.
- All members inherit the group's permissions.

When a user belongs to multiple groups, permissions are merged — the highest level wins.

4.4.4 Permission caching

The `PermissionsCache` is a singleton in-memory dictionary:

- **Key:** user ID.
- **Value:** dict mapping module slugs to access level names.
- **Invalidation:** triggered when group membership or role assignment changes (calling `permissions_cache.invalidate(user_id)` or `permissions_cache.invalidate()` for global clear).

This avoids repeated database queries for permission checks on every request. The cache lives in the application process — in multi-worker deployments, each worker maintains its own cache.

4.4.5 Admin bypass

Users with the `Admin` role bypass all permission checks. Admin status is checked before the permission resolution chain runs.

4.4.6 API permissions

API tokens inherit the owning user's permissions. The `check_token()` before-request hook resolves the user from the token and applies the same RBAC checks.

Copyright © 2024–2026 pixelotes — Elastic License

4.5 Architecture Decision Records

This page documents the key architectural decisions made during OpsDeck's development, the context behind them, and the trade-offs accepted.

4.5.1 ADR-001: Flask as web framework

Context. OpsDeck needed a Python web framework that supports rapid feature development, has a mature ecosystem, and doesn't impose unnecessary structure.

Decision. Flask was chosen over Django, FastAPI, and other alternatives.

Rationale.

- Flask's blueprint system maps cleanly to OpsDeck's modular structure (one blueprint per functional area).
- No ORM opinions — SQLAlchemy was chosen independently for its flexibility.
- Lightweight core with extensions for exactly what's needed (Flask-Login, Flask-WTF, flask-talisman, Flask-Limiter).
- Django's batteries-included approach would have imposed an admin interface, user model, and migration system that conflict with OpsDeck's custom requirements.
- FastAPI was considered but the project predates its maturity and OpsDeck's server-rendered templates don't benefit from async.

Trade-offs. More manual wiring than Django. No built-in admin panel. Extension quality varies.

4.5.2 ADR-002: Monolithic architecture

Context. The application covers IT ops, compliance, procurement, CRM, and more — domains that could theoretically be separate services.

Decision. Ship as a single deployable monolith.

Rationale.

- Single operator (or very small team) — operational overhead of microservices is unjustifiable.
- Cross-domain queries (e.g., "show me all assets linked to this compliance control that belong to this vendor") are trivial with a shared database and impossible with service boundaries.
- Deployment is `docker-compose up` or a single Helm release. No service mesh, no API gateway, no distributed tracing required.
- The internal module structure (models/routes/services) provides logical separation without physical separation.

Trade-offs. All modules scale together. A bug in one area can affect others. The codebase must stay well-organized as it grows.

4.5.3 ADR-003: Server-rendered templates over SPA

Context. Modern web apps often use React/Vue/Angular with a separate API backend.

Decision. Use Jinja2 server-side rendering with progressive enhancement via vanilla JavaScript.

Rationale.

- Eliminates an entire build toolchain (Webpack/Vite, Node.js CI, API versioning).
- Forms, tables, and CRUD operations — OpsDeck's primary UI patterns — are faster to build with server rendering.
- No API contract negotiation between frontend and backend teams (there is no frontend team).
- Bootstrap 5 provides responsive layout, DataTables handles sortable tables, Tom Select handles enhanced selects, Chart.js handles dashboards.
- The REST API exists for programmatic access and integrations, not for powering the UI.

Copyright © 2024–2026 pixelotes — Elastic License

Trade-offs. Less interactive than a SPA. Page transitions require full reloads (mitigated by fast server responses). Complex client-side interactions (e.g., drag-and-drop UAR configuration) require vanilla JS that could be cleaner in React.

4.5.4 ADR-004: Elastic License

Context. OpsDeck needs to be open-source enough for transparency, self-hosting, and community contributions, but protected against cloud providers offering it as a managed service.

Decision. Elastic License 2.0.

Rationale.

- Allows free use, modification, and self-hosted deployment.
- Prevents third parties from offering OpsDeck as a managed SaaS without contributing back.
- Well-precedented — used by Elasticsearch, Kibana, and other infrastructure software.
- Simpler than AGPL for end users who just want to self-host.

Trade-offs. Not OSI-approved "open source." Some organizations' legal teams may require review. Cannot be included in Linux distributions that require OSI licenses.

4.5.5 ADR-005: APScheduler over Celery

Context. OpsDeck needs background job scheduling for UAR execution, compliance drift snapshots, renewal notifications, and data retention.

Decision. APScheduler (in-process) instead of Celery with Redis/RabbitMQ.

Rationale.

- Zero additional infrastructure — no Redis, no RabbitMQ, no separate worker process.
- Jobs run within the Flask application context with full access to the database and configuration.
- Scheduled jobs (cron-style) are the primary use case, not high-throughput task queues.
- Job definitions live in Python code alongside the services they invoke, not in a separate configuration.

Trade-offs. Jobs run in the application process — a long-running job blocks that worker. No horizontal scaling of job execution. No built-in retry/dead-letter queue (manual error handling in each job).

4.5.6 ADR-006: PostgreSQL as sole database

Context. The application needs a relational database. Multiple databases were considered.

Decision. PostgreSQL only. MySQL is not officially supported.

Rationale.

- Robust JSON support for storing custom properties and audit diffs.
- Advanced indexing (GIN, GiST) for full-text search without an external search engine.
- Mature ecosystem with excellent tooling (pg_dump, pg_restore, pgAdmin).
- Well-supported by all deployment targets (Docker, Kubernetes, RDS, Aurora).

Trade-offs. Limits deployment options for organizations that standardize on MySQL/MariaDB. The psycopg2-binary dependency requires PostgreSQL client libraries.

Copyright © 2024–2026 pixelotes — Elastic License

4.5.7 ADR-007: Sequential migration numbering

Context. Alembic (via Flask-Migrate) generates random hex revision IDs by default.

Decision. Use sequential numbering (`001_initial.py` , `002_add_notes_field.py` , etc.).

Rationale.

- Human-readable ordering — you can see at a glance which migration came first.
- Easier to discuss in code reviews ("migration 015" vs "migration a3f7b2c1").
- Linear history enforced — no branching migrations that could conflict.

Trade-offs. Requires manual coordination if multiple developers create migrations simultaneously. Merge conflicts on migration numbers must be resolved manually.

4.5.8 ADR-008: Bootstrap 5 + vanilla JS over React/Vue

Context. The frontend needs interactive components (sortable tables, enhanced selects, charts, calendar views, org charts).

Decision. Bootstrap 5 as the CSS framework, with purpose-specific JS libraries instead of a framework.

Rationale.

- Bootstrap 5 dropped jQuery dependency — clean, modern CSS with responsive grid.
- Each interactive need is solved by the best-in-class library for that specific problem: Simple DataTables for tables, Tom Select for selects, Chart.js for charts, FullCalendar for calendar views, Mermaid for diagrams, SortableJS for drag-and-drop, OrgChart.js for org charts.
- No build step required — `copy-assets.js` copies vendor files from `node_modules` to `static/vendor/` .
- Total JS bundle is smaller than any SPA framework's baseline.

Trade-offs. No component model — UI reuse is via Jinja2 includes and macros, which are less composable than React components. State management across related UI elements requires manual DOM manipulation.

4.5.9 ADR-009: WeasyPrint for PDF generation

Context. OpsDeck needs to generate PDF reports (audit exports, compliance reports, asset inventories).

Decision. WeasyPrint — HTML/CSS to PDF rendering in Python.

Rationale.

- Uses the same Jinja2 templates and CSS already written for the web UI — no separate template language.
- Pure Python (with system library dependencies for Pango/Cairo) — no headless browser required.
- Produces high-quality PDFs with proper typography, tables, and page breaks.

Trade-offs. Requires system packages (`libpango` , `libcairo`) which complicate the Docker image. Rendering complex layouts can be slow for large reports. CSS support is not 100% — some flexbox/grid features are unavailable.

4.5.10 ADR-010: DeepDiff for UAR and audit change detection

Context. The UAR engine and audit logging need to compare complex objects and produce human-readable diffs.

Decision. DeepDiff library for structured comparison.

Copyright © 2024–2026 pixelotes — Elastic License

Rationale.

- Handles nested dict/list comparison out of the box.
- Produces structured output (added, removed, changed) that can be stored as JSON in the database.
- Used both in the UAR engine (comparing user records between systems) and in the audit listener (detecting which fields changed on a model update).
- Mature library with good edge-case handling (type changes, list reordering, etc.).

Trade-offs. Can be slow on very large objects. Output format requires post-processing for human display. Library size is non-trivial for what is conceptually a simple operation.

Copyright © 2024–2026 pixelotes — Elastic License

4.6 Dependencies

Complete list of Python and JavaScript dependencies used by OpsDeck, with version, purpose, and justification.

4.6.1 Python dependencies

Production (`requirements.txt`)

Package	Version	Purpose
Flask	3.1.3	Web framework — lightweight, blueprint-based, extension-friendly
Flask-SQLAlchemy	3.1.1	SQLAlchemy integration for Flask — session management, model base class
Flask-Migrate	4.1.0	Database migrations via Alembic — schema versioning and upgrades
Flask-Login	0.6.3	Session-based user authentication — login/logout, remember-me, session protection
Flask-Dance	7.1.0	OAuth provider integration — Google SSO support
Flask-WTF	1.2.2	Form handling with CSRF protection — validation, rendering, file uploads
flask-talisman	1.1.0	Security headers — CSP, HSTS, X-Frame-Options, referrer policy
flask-smorest	0.46.2	REST API framework — OpenAPI spec generation, request/response validation
Flask-Limiter	4.1.1	Rate limiting — brute-force protection, API throttling
marshmallow-sqlalchemy	1.4.2	Auto-schema generation from SQLAlchemy models for API serialization
SQLAlchemy	*(transitive)*	ORM — model definition, querying, relationship management
Alembic	*(transitive)*	Migration engine — auto-detect model changes, generate migration scripts
psycopg2-binary	2.9.11	PostgreSQL driver — binary distribution, no compilation needed
APScheduler	3.11.2	Background job scheduler — cron-style jobs, in-process execution
gunicorn	25.1.0	Production WSGI server — multi-worker, pre-fork model
requests	2.32.5	HTTP client — webhook delivery, external API calls
python-dateutil	2.9.0	Date parsing and manipulation — relative deltas, recurring dates
python-dotenv	1.2.2	Environment variable loading from `.env` files
pytz	2026.1	Timezone definitions — IANA timezone database
MarkupSafe	3.0.3	HTML escaping — Jinja2 dependency, XSS prevention
Markdown	3.10.2	Markdown rendering — documentation pages, rich text descriptions
WeasyPrint	68.1	HTML-to-PDF rendering — report generation using existing templates
ecs-logging	2.3.0	ECS-format structured logging — Elasticsearch-compatible log format
deepdiff	8.6.1	Structured object comparison — UAR engine, audit change detection
slack_sdk	3.41.0	Slack API client — notification delivery to Slack channels
PyJWT	2.12.0	JWT token handling — OAuth flow, token validation
cryptography	46.0.5	Cryptographic operations — Fernet encryption for credential vault
boto3	1.42.67	AWS SDK — S3 storage for attachments in cloud deployments

Copyright © 2024–2026 pixelotes — Elastic License

Development (`requirements-dev.txt`)

Package	Version	Purpose
pytest	9.0.2	Test framework — 377 tests across all modules
Faker	40.8.0	Fake data generation — test fixtures, demo data seeding

4.6.2 JavaScript dependencies

Production (`package.json`)

Package	Version	Purpose
bootstrap	5.3.x	CSS framework — responsive grid, components, utilities
@popperjs/core	2.11.x	Tooltip/popover positioning — Bootstrap dependency
@fortawesome/fontawesome-free	7.2.x	Icon library — UI icons throughout the application
chart.js	4.5.x	Chart library — dashboard visualizations, compliance trends
simple-datatables	10.2.x	Table enhancement — sort, search, paginate HTML tables
tom-select	2.5.x	Enhanced select inputs — search, tagging, remote data
fullcalendar	6.1.x	Calendar component — scheduling views, date-based navigation
mermaid	11.13.x	Diagram renderer — service topology, flowcharts from text
suneditor	2.47.x	WYSIWYG editor — rich text editing for descriptions and notes
easymde	2.20.x	Markdown editor — documentation editing with preview
sortablejs	1.15.x	Drag-and-drop — reorderable lists (onboarding items, priorities)
swagger-ui-dist	5.32.x	API documentation UI — interactive endpoint testing
html2canvas	1.4.x	DOM-to-canvas screenshot — client-side report captures
orgchart.js	0.0.4	Organization chart — hierarchical user visualization

4.6.3 Dependency management

Python dependencies are pinned to exact versions (`==`) in `requirements.txt` for reproducible builds. The `update-deps.sh` script automates the update workflow:

1. Creates a fresh virtual environment.
2. Installs packages with latest compatible versions.
3. Freezes to `requirements.txt` .
4. Runs the test suite to verify compatibility.

JavaScript dependencies are managed via npm. `copy-assets.js` copies distribution files from `node_modules` to `src/static/vendor/` — no bundler involved.

Copyright © 2024–2026 pixelotes — Elastic License

4.7 Frontend Stack

OpsDeck uses server-rendered Jinja2 templates with Bootstrap 5, enhanced by purpose-specific JavaScript libraries. There is no SPA framework, no build step, and no JS bundler.

4.7.1 Template architecture

Templates follow Jinja2's layout inheritance pattern:

```
templates/
├── layout.html              # Base layout: navbar, sidebar, footer, JS/CSS includes
├── components/              # Reusable includes: modals, forms, tables, pagination
│   ├── confirm_modal.html
│   ├── pagination.html
│   ├── compliance_links.html
│   └── ...
└── [module]/                # One directory per functional module
    ├── list.html              # Table listing with filters
    ├── detail.html            # Entity detail view
    ├── form.html              # Create/edit form
    └── ...
```

All module templates extend `layout.html` and use `{% block content %}` for page-specific content. Reusable components are included via `{% include 'components/...' %}`.

4.7.2 Bootstrap 5

Bootstrap 5 (loaded from `static/vendor/`) provides:

- Responsive grid system for layout.
- Form styling and validation states.
- Card components for detail views.
- Table styling (enhanced by Simple DataTables).
- Modal dialogs for confirmations and quick actions.
- Toast notifications for feedback messages.

No custom Bootstrap build — the full distribution is used via `copy-assets.js`.

Copyright © 2024–2026 pixelotes — Elastic License

4.7.3 JavaScript libraries

Each interactive need is solved by a purpose-specific library:

Library	Version	Purpose	Where used
Simple DataTables	10.2.x	Sortable, searchable, paginated tables	All list views
Tom Select	2.5.x	Enhanced select inputs with search and tagging	Entity selection dropdowns
Chart.js	4.5.x	Dashboard charts and visualizations	Dashboard, reports, compliance drift
FullCalendar	6.1.x	Calendar views for scheduling	Maintenance, renewals, training
Mermaid	11.13.x	Diagram rendering from Markdown-like syntax	Service topology, documentation
SunEditor	2.47.x	Rich text editor (WYSIWYG)	Description fields, notes
EasyMDE	2.20.x	Markdown editor	Documentation pages
SortableJS	1.15.x	Drag-and-drop reordering	Onboarding pack items, priority lists
Swagger UI	5.32.x	Interactive API documentation	`/swagger-ui` endpoint
html2canvas	1.4.x	Client-side screenshots	Report generation
OrgChart.js	0.0.4	Organization chart rendering	People → Organization
Font Awesome	7.2.x	Icons throughout the UI	Global

4.7.4 Asset pipeline

There is no Webpack, Vite, or other bundler. The asset pipeline is a simple Node.js script:

```
npm install       # Install packages to node_modules/
npm run build-assets  # Copies vendor files to src/static/vendor/
```

`copy-assets.js` reads `package.json` dependencies and copies the required distribution files (minified JS and CSS) to `src/static/vendor/`. Templates load these files with standard `<script>` and `<link>` tags.

4.7.5 JavaScript patterns

Client-side JavaScript follows these conventions:

- **No module bundling.** Scripts are loaded globally via `<script src="...">` tags.
- **Progressive enhancement.** Core functionality works without JavaScript. JS adds interactivity (search, sort, AJAX updates) on top.
- **AJAX for partial updates.** Some actions (status toggles, inline edits, bulk operations) use `fetch()` to call route endpoints and update the DOM without full page reloads.
- **Event delegation.** Table-level event handlers manage clicks on dynamically rendered rows.

Copyright © 2024–2026 pixelotes — Elastic License

4.8 API Design

The REST API is built with flask-smorest, which provides OpenAPI (Swagger) spec generation, request validation, and response serialization.

4.8.1 Architecture

```
src/api.py          # Blueprint registration, token auth hook
src/schemas.py      # Marshmallow schemas (auto-generated from SQLAlchemy models)
```

The API blueprint is registered at `/api/v1/` with a `before_request` hook that validates bearer tokens on every request.

4.8.2 REST conventions

Verb	Usage
`GET /api/v1/{resource}`	List entities (paginated)
`GET /api/v1/{resource}/{id}`	Get single entity
`POST /api/v1/{resource}`	Create entity (writable resources only)

Currently, most resources are read-only (GET). Writable endpoints exist for changes, incidents, and onboarding processes.

4.8.3 Authentication

Bearer token in the `Authorization` header. The `check_token()` hook:

1. Extracts the token from `Authorization: Bearer <token>`.
2. Looks up the user by token value.
3. Sets `g.api_user` for the request context.
4. Returns 401 if the token is missing or invalid.
5. Logs all access attempts (success and failure) in ECS format.

4.8.4 Marshmallow schemas

Schemas are auto-generated from SQLAlchemy models using `marshmallow-sqlalchemy`:

```
class AssetSchema(BaseSchema):
    custom_properties = fields.Dict(dump_only=True)
    class Meta(BaseSchema.Meta):
        model = Asset
```

This provides:

- Automatic field mapping from model columns.
- Exclusion of sensitive fields (e.g., `password_hash`, `api_token` on UserSchema).
- JSON serialization of complex types (dates, enums, relationships).
- Custom properties exposed as a flat dict.

4.8.5 OpenAPI spec

flask-smorest auto-generates an OpenAPI 3.0 spec from the registered blueprints and schemas. Available at:

- `/openapi.json` — raw spec.
- `/swagger-ui` — interactive documentation UI (Swagger UI).

Copyright © 2024–2026 pixelotes — Elastic License

4.8.6 Pagination

List endpoints accept `page` and `page_size` query parameters. Responses include pagination metadata:

```
{
  "items": [...],
  "total": 142,
  "page": 1,
  "page_size": 25,
  "pages": 6
}
```

Default page size is 25. Maximum page size is 100.

Copyright © 2024–2026 pixelotes — Elastic License

4.9 Scheduler & Jobs

OpsDeck uses APScheduler (Advanced Python Scheduler) for background job execution within the Flask application process. No external queue or worker infrastructure is required.

4.9.1 Configuration

APScheduler runs in-process with the Flask application. It initializes during app startup and shares the application context, database session, and configuration.

Jobs are defined programmatically in the application factory (`src/__init__.py`) and service modules. The scheduler uses a thread pool executor with a single job store (in-memory by default).

> **Note**
>
> Because the scheduler runs in-process, jobs execute serially within the thread pool. A long-running job will not block the web server (Gunicorn spawns separate workers), but it will delay other scheduled jobs.

4.9.2 Registered jobs

Job	Schedule	Service	Description
Compliance drift snapshot	Daily	`compliance_drift_service`	Captures current compliance status for all frameworks and compares with the previous snapshot to detect regressions
UAR scheduled execution	Configurable per comparison	`uar_service`	Loads datasets, runs the comparison engine, and generates findings
Subscription renewal alerts	Daily	`notifications`	Checks for subscriptions approaching renewal and sends notifications
Certificate expiry alerts	Daily	`notifications`	Checks for certificates approaching expiry date
Data retention	Weekly	—	Archives old records per configured retention policies

4.9.3 Compliance drift detection

The drift detection job is the most complex scheduled task:

1. **Snapshot capture.** Queries all `FrameworkControl` records and their current compliance status (derived from linked evidence). Stores the snapshot as a JSON record with timestamp.
2. **Drift analysis.** Compares the new snapshot with the most recent previous snapshot. For each control, detects status changes.
3. **Classification.** Changes are classified as:
   - **Regression** — status worsened (e.g., Compliant → Warning). Assigned severity: critical for Non-compliant, high for Warning, medium for other regressions.
   - **Improvement** — status improved (e.g., Non-compliant → Compliant).
4. **Alerting.** Critical regressions are logged and (when configured) trigger notifications via email or Slack webhook.

The drift timeline is visualized on the Compliance Drift dashboard, showing changes over configurable periods (7, 30, 90 days).

Copyright © 2024–2026 pixelotes — Elastic License

4.9.4 UAR scheduled execution

UAR comparisons can be scheduled at the comparison level:

1. The scheduler checks for comparisons due for execution based on their configured frequency (daily, weekly, monthly).
2. For each due comparison, it triggers `uar_service.execute_comparison()`.
3. The engine loads both datasets, performs the DeepDiff comparison, and generates `UARFinding` records.
4. Progress is logged for long-running comparisons.
5. On completion, the execution record is updated with finding counts and status.

4.9.5 Error handling

- Each job wraps its execution in a try/except block.
- Errors are logged with full stack traces using the ECS-format structured logger.
- Failed jobs do not crash the scheduler — the next scheduled run proceeds normally.
- There is no automatic retry mechanism — failed runs must be investigated from the logs or re-triggered manually.

4.9.6 Manual execution

All scheduled jobs can be triggered manually:

- **Compliance drift snapshot:** click "Capture Snapshot" on the Compliance Drift page.
- **UAR execution:** click "Run Now" on the comparison detail page.
- **Via CLI:** `flask run-job <job_name>` (if configured).

Copyright © 2024–2026 pixelotes — Elastic License

4.10 Security Model

This document describes OpsDeck's security architecture, covering authentication, authorization, data protection, and security headers.

4.10.1 Authentication flow

OpsDeck supports two authentication methods: local credentials and Google OAuth via Flask-Dance.

Local authentication

1. User submits email and password.
2. Password is verified against the stored hash (Werkzeug PBKDF2).
3. If MFA is enabled (`MFA_ENABLED=True`), a TOTP token is required.
4. Flask-Login creates a server-side session with secure cookie flags.
5. The user's known IP is recorded (`UserKnownIP` model) for anomaly detection.

OAuth (Google SSO)

1. User clicks "Sign in with Google."
2. Flask-Dance redirects to Google's OAuth consent screen.
3. On callback, the email is matched to an existing user record.
4. If the user exists, a session is created. If not, access is denied (no self-registration).

> **Warning**
>
> `OAUTHLIB_INSECURE_TRANSPORT=1` must only be set in development. It disables the HTTPS requirement for OAuth callbacks.

4.10.2 Session management

Setting	Default	Production
`SESSION_COOKIE_SECURE`	`False`	`True`
`SESSION_COOKIE_HTTPONLY`	`True`	`True`
`SESSION_COOKIE_SAMESITE`	`Lax`	`Lax` or `Strict`

Sessions are stored server-side. The cookie contains only the session identifier, signed with `SECRET_KEY` .

4.10.3 Authorization (RBAC)

Role hierarchy

Role	Access
Admin	Full system access, user management, configuration
Manager	Read/write to operational data, limited admin functions
User	Read access to assigned data, limited write access
API	Programmatic access via bearer token (inherits user's permissions)

Copyright © 2024–2026 pixelotes — Elastic License

Permission resolution

Permissions are defined at the module level. Each module (assets, compliance, risk, etc.) has three permission levels: `can_read`, `can_write`, `can_delete`.

Permission checks follow this chain:

1. Route decorator `@permission_required(module, level)` intercepts the request.
2. `permissions_service.py` resolves the user's effective permissions by combining their direct role with group-inherited permissions.
3. Resolved permissions are cached in-memory (`permissions_cache.py`) to avoid repeated DB queries.
4. Cache is invalidated when group membership or role assignment changes.

API authentication

API endpoints accept a bearer token in the `Authorization` header. Tokens are generated per-user, hashed with SHA-256 before storage, and carry the same permissions as the owning user. Tokens can be revoked from the user profile.

4.10.4 Data protection

Credential encryption

The credentials vault (`models/credentials.py`) uses Fernet symmetric encryption (from the `cryptography` library) to encrypt secret values at rest. The encryption key is derived from `SECRET_KEY`. Each `CredentialSecret` record stores a Fernet token that can only be decrypted with the application's key.

Password hashing

User passwords are hashed using Werkzeug's `generate_password_hash` (PBKDF2 with SHA-256, 600,000 iterations by default). Plaintext passwords are never stored or logged.

API token hashing

API tokens are displayed once at creation time. The stored value is a SHA-256 hash — the original token cannot be recovered from the database.

4.10.5 Security headers (flask-talisman)

flask-talisman enforces:

- `Strict-Transport-Security` (HSTS)
- `Content-Security-Policy` (CSP)
- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `Referrer-Policy: strict-origin-when-cross-origin`

CSP is configured to allow the specific CDN sources used by the frontend libraries (Bootstrap, Chart.js, etc.) while blocking inline scripts where possible.

4.10.6 Rate limiting

Flask-Limiter protects against brute-force and abuse:

- Login endpoint: rate-limited to prevent credential stuffing.
- API endpoints: per-token rate limits.
- Configurable via environment variables.

Copyright © 2024–2026 pixelotes — Elastic License

4.10.7 Audit logging

Every data mutation is captured by the SQLAlchemy event listener in `utils/audit_listener.py`:

- **Who**: User ID from the Flask-Login session.
- **What**: Table name, record ID, operation type (INSERT/UPDATE/DELETE).
- **When**: UTC timestamp.
- **Details**: JSON diff of changed fields (via DeepDiff).

Audit log records are append-only — there is no UI or API to delete them. They serve as the immutable audit trail required by ISO 27001 and SOC 2.

Copyright © 2024–2026 pixelotes — Elastic License

5. Frequently Asked Questions

5.1 General

What is OpsDeck?

OpsDeck is an integrated IT operations and governance platform that consolidates asset management, compliance, vendor management, risk management, and service catalog into a single self-hosted application. It's designed for IT teams in regulated environments who need operational rigor without enterprise complexity.

Who is OpsDeck for?

Mid-market IT teams (50–500 employees) in finance, healthcare, SaaS, and other regulated industries. It's particularly well-suited for organizations where a single person or small team manages IT operations, security, and compliance simultaneously.

What compliance frameworks does OpsDeck support?

OpsDeck ships with support for ISO 27001 and SOC 2, and supports custom frameworks. You can define any framework with any number of controls. The compliance linking system is framework-agnostic — any entity can be linked to any control as evidence.

5.2 Licensing

What license does OpsDeck use?

Elastic License 2.0. You can use, modify, and self-host OpsDeck freely. The restriction is on offering it as a managed service (SaaS) to third parties.

Can I use OpsDeck internally at my company?

Yes, without restriction. The Elastic License only restricts offering OpsDeck as a managed service to others.

Is OpsDeck open source?

The source code is publicly available and modifiable, but the Elastic License is not OSI-approved. If your organization requires OSI-approved licenses, check with your legal team.

5.3 Deployment

What are the minimum requirements?

Python 3.11+, PostgreSQL 15+, and approximately 1 GB of RAM for the application. For Docker deployments, Docker Engine 24+ and Docker Compose v2.

Does OpsDeck support MySQL?

PostgreSQL is the only officially supported database. The codebase uses PostgreSQL-specific features (JSON fields, advanced indexing). MySQL may work for basic functionality but is not tested.

Can I run multiple instances for high availability?

Yes, with external PostgreSQL and shared storage for attachments. Multiple application replicas can run behind a load balancer. Ensure `SECRET_KEY` is identical across all instances.

Does OpsDeck need internet access?

No. OpsDeck is fully self-contained and can run in air-gapped environments. The only external connections are optional: SMTP for email, Slack webhook for notifications, and Google OAuth for SSO.

Copyright © 2024–2026 pixelotes — Elastic License

5.4 Features

Can I import existing data?

Yes. OpsDeck includes a CLI for bulk CSV import of users, assets, suppliers, and contacts. See the Data Import guide.

Does OpsDeck have an API?

Yes. A REST API with bearer token authentication covers all major entities. Interactive documentation is available at `/swagger-ui` on your deployment. See API Usage.

Can I customize the fields on assets and users?

Yes. Custom properties (key-value custom fields) can be defined at the organization level and applied to assets, users, and peripherals. See Configuration.

How does the credential vault work?

Credentials are stored encrypted at rest using Fernet symmetric encryption (from the `cryptography` library). The encryption key is derived from your `SECRET_KEY`. Access to view secrets is controlled by RBAC permissions.

5.5 Troubleshooting

The application won't start — database connection error.

Verify your `DATABASE_URL` environment variable. Ensure PostgreSQL is running and accessible. For Docker deployments, check that the `db` service is healthy before the `web` service starts (`depends_on` with `condition: service_healthy`).

Migrations fail after upgrading.

Run `flask db upgrade` manually to see detailed error output. If a migration fails midway, you may need to resolve the database state manually. Check the Database Management guide.

I forgot the admin password.

Use the CLI: `flask reset-password admin@example.com`. This resets the password and prompts for a new one on next login.

Search returns no results.

Universal search indexes entity names, descriptions, serial numbers, and notes. Ensure the entities you're looking for have searchable content in those fields. Custom field values are also indexed.

Copyright © 2024–2026 pixelotes — Elastic License

6. Changelog

The full changelog is maintained in the main repository:

:octicons-mark-github-16: View Changelog on GitHub

This file tracks all notable changes to OpsDeck, organized by version.

Copyright © 2024–2026 pixelotes — Elastic License